Department of Statistics

DEPARTMENT OF STATISTICS 2025/26

Submitted for the Master of Science, London School of Economics, University of London

---

ST498 Capstone Project

An Empirical Analysis of Multi-Agent Systems for Football Data Analytics and Performance Prediction

---

Candidate numbers:

73639
59489
66190

Supervisors:

Dr Marcos Barreto (LSE)
Dr Huma Lodhi (Sky)

August 4, 2026

Contents

Executive Summary **vi**

1 Introduction **1**
1.1 Technological Ecosystem . . . 1
1.2 Football Outcomes . . . 1
1.3 Research Focus and Objectives . . . 2
1.4 Main Contributions (Results) . . . 2
1.5 Report Structure . . . 2

2 Literature Review (Related Work) **3**
2.1 Evaluation gaps in multi-agent LLM systems and orchestration design patterns 3
2.2 Multimodal vs. structured-query approach in agentic systems for football . . . 3
2.3 Calibration and Evaluation practices in match-outcome prediction . . . 3
2.4 Memory and Retrieval in Conversational Agents . . . 4

3 Data **5**
3.1 Data Sources . . . 5
3.2 Data Engineering Pipeline . . . 7
3.2.1 Extraction and Raw Staging . . . 7
3.2.2 Source-Specific Cleaning and Transformation . . . 7
3.2.3 Entity Resolution and Cross-Source Integration . . . 8
3.2.4 Validation and Database Loading . . . 8
3.2.5 Serving Tables and Model-Ready Features . . . 9
3.3 Knowledge Base . . . 9
3.4 Exploratory Data Analysis . . . 10
3.4.1 Match-Outcome Composition . . . 10
3.4.2 Goal-Scoring Profile . . . 11
3.4.3 Feature Availability . . . 12
3.4.4 Implications for Downstream Analytics . . . 12
3.5 Schema Documentation: YAML Specification . . . 13

4 Methodology **14**
4.1 Multi-Agent System Architecture . . . 14
4.2 Memory Architecture . . . 16
4.3 Services Used . . . 19
4.4 Tools . . . 20
4.5 Prediction Node . . . 21
4.5.1 Match-Outcome Algorithm . . . 24

5 Experiment Designs **26**
5.1 Setup of single LLM agents . . . 26
5.2 Ground truth dataset and evaluation process . . . 26
5.3 Golden-Set Questions (data analysis, $n = 30$) . . . 27
5.3.1 Experiment 1: Testing different degrees of agent autonomy . . . 30
5.3.2 Experiment 2: Testing MAS vs both single LLM variants . . . 30
5.3.3 Experiment 3: Testing memory augmented MAS . . . 30

6 Results **32**
6.1 MAS Autonomy Comparison . . . 32
6.2 MAS vs Single-LLM Agents . . . 33
6.3 Memory Augmented MAS Results . . . 35
6.3.1 SQL Cache Threshold Stress Test . . . 35
6.3.2 Session Based Memory Ablation . . . 36
6.4 Prediction Backtest Results . . . 37

7 Limitations **38**

8 Conclusion **41**

A Appendix: Experimental Setup and Results **42**
A.1 Model IDs and Configuration . . . 42
A.2 Failure Taxonomy (semi-controlled variant) . . . 42
A.3 Root-Cause Analysis of Failures . . . 43
A.4 Exact Prompts per Condition . . . 44
A.4.1 Shared preamble (all agents) . . . 44
A.4.2 MAS Manager (intent + entity extraction) . . . 44
A.4.3 MAS Data Discovery . . . 44
A.4.4 MAS Data Analyst, CONTROLLED (template only) . . . 44
A.4.5 MAS Data Analyst, SEMI (route template-or-SQL) . . . 44
A.4.6 MAS Data Analyst, AUTONOMOUS (free SQL + self-debate) . . . 45
A.4.7 MAS Synthesiser (data questions) . . . 45
A.4.8 MAS Prediction synthesiser . . . 45
A.4.9 Single-LLM + tools (system prompt) . . . 45
A.4.10 Single-LLM + raw data (system prompt) . . . 46
A.4.11 Single-LLM + raw data (prediction variant) . . . 46
A.4.12 Tool: predict_match_outcome (schema shown to the model) . . . 46
A.4.13 Template catalogue (controlled / semi) . . . 46
A.5 Caveats . . . 47

References **48**

List of Figures

3.1 Primary and supporting contributions of each provider to the integrated knowledge base. Empty cells indicate that the provider is not used for the corresponding information family. 6

3.2 Layered data architecture for grounded football analytics. Raw provider records are progressively transformed into canonical entities, task-ready serving relations, and evidence accessible to the analytical system. 7

3.3 Overall composition of home wins, draws, and away wins in the frozen analytical extract. The figure reports relative shares rather than absolute match counts. 11

3.4 Normalised distribution of total goals per match in the frozen analytical extract. Matches with five or more total goals are grouped in the rightmost category. . 11

3.5 Aggregated feature-availability profile by information family in the frozen analytical extract. A match is complete for a family only when all fields assigned to that family are non-missing. 12

4.1 The LangGraph orchestration graph. The diamond denotes the Manager's routing decision; edge labels give the intent value(s) governing each branch. Data Analyst has a second, internal routing split (`full_pipeline` vs. `data_request`/error) determining whether Visualizer is invoked before Synthesizer. 14

4.2 Functional taxonomy of agent memory, following Hu et al. (2026), mapped onto MA-FDSS's memory stores. `documents` is drawn as a separate root rather than a child of Agent Memory, reflecting its classification as retrieval-augmented generation over a static knowledge base rather than agent memory proper. . . 17

4.3 Capability selection cascade: the three-step fallback sequence resolving which prediction capability runs for a given query. 23

4.4 The prediction agent within the MAS pipeline: memory retrieval, manager intent classification, and the prediction-specific dispatch, execution, and post-processing stages, drawing on offline-trained model artefacts. 23

List of Tables

3.1 Principal serving views in the knowledge base. 10

4.1 The four `pgvector` tables. Similarity is cosine on 384-dimensional embeddings; thresholds are applied after retrieval. 18
4.2 Prediction capability families. 22

5.1 Golden-set data-analysis questions ($n = 30$), with category, difficulty tier, and template coverage. IDs are inherited from the original 46-question set from which this benchmark was curated; gaps reflect questions not retained. 27
5.2 Prediction (forecast) questions ($n = 30$) with realised outcome and model ground-truth probabilities (H/D/A). 29

6.1 Accuracy by degree of Data Analyst autonomy, split by SQL-template coverage (46 golden-set data-analysis questions; 28 in-coverage, 18 out-of-coverage). . . 32
6.2 MAS vs. single-LLM baselines on the 30 data-analysis golden-set questions. . 33
6.3 Delegation to the prediction agent: MAS vs. single-LLM baselines (30 prediction questions). 34
6.4 Three tier SQL cache sweep on the 46 question golden set (0.88 cosine threshold). 35
6.5 Session based memory ablation: paired accuracy change from enabling each store, measured on beneficiary turns (memory on arm minus memory off arm). Positive values favour memory. 36
6.6 Out-of-sample match outcome prediction, 2024–2025 season ($n = 379$). 37

A.1 Model identifiers and temperature configuration per logical family. 42
A.2 Failure taxonomy for the semi-controlled variant, by model and failure mode (autonomy experiment, $n = 46$). 43

Executive Summary

Large Language Models (LLMs) have become the modern interface layer for data-driven analysis, letting non-technical users turn natural-language questions into database queries. Football is a natural domain for this: fans, journalists, and analysts draw on the same rich structured records whether they want last season's best goal difference or a forecast for an upcoming fixture. This project harnesses AI and agentic systems to answer complex football queries, such as match analysis, strategy, and match-outcome or league-rank predictions. Given that football analytics is dynamic and multi-player, we build a multi-agent system (MAS) to test whether, and when, a multi-agent architecture is worth its cost relative to a single LLM given equivalent resources.

The challenge this paper investigates is non-trivial for several reasons. Research suggests MAS achieve superior accuracy on complex, multi-step tasks, but that evidence rests on general-purpose reasoning benchmarks that do not isolate orchestration from tool and structured-database access, nor report the token and latency cost orchestration incurs. A fair test therefore requires a controlled design that separates these effects rather than crediting the whole improvement to orchestration. Second, football data is heterogeneous: different sources assign unique identifiers to players, clubs, and matches, so the system must reconcile entities before any agent can query them reliably. Third, predictive questions require a genuine calibrated model rather than free-text reasoning, because users need to trust the probabilities displayed. Finally, a production system must handle unseen queries, not only those tested during evaluation. Each of these constraints shapes a design decision reported below, and together they motivate a controlled comparison rather than a headline benchmark score.

To address this we built a knowledge base: a centralised store of structured and unstructured data, organised as a three-layer relational schema that separates raw ingestion, entity standardisation, and query-serving. Separating these layers means source-specific quirks are absorbed at ingestion and never leak into the tables the agents actually query. The knowledge base spans ten European top-flight competitions across twelve seasons (2014–15 to 2025–26), covering roughly six thousand players. The match-outcome model reported here is trained on Premier League matches from 2020–2024 (1,502 fixtures) and evaluated out-of-sample on 2024–2025 (379 fixtures). Deterministic mappings resolve inconsistent naming across sources, and manual validation maintains a machine-readable schema specification. Tables, columns, key constraints, and named join paths are injected into the agents' context so they never reverse-engineer the database at query time.

LangGraph was used to orchestrate the pipeline as a directed graph. A manager decomposes the query intent and extracts structured parameters (teams, seasons, metrics), routing deterministic questions to a data-analysis branch and forward-looking questions to a prediction agent. A Data Discovery agent selects the correct tables; a Data Analyst agent generates SQL behind a read-only guardrail; and a Synthesizer agent composes the natural-language answer. Before any result reaches the user, an evaluation agent verifies it against the retrieved figures.

The system is fully autonomous, built around a bounded retry loop that grants two attempts before capping effort to contain cost, with generated SQL assessed in a self-debate step before an answer is returned. A persistent memory layer lets the MAS build on what it has already done rather than starting each turn cold: a conversation history carries context across turns, an SQL cache reuses prior query work, and a correction store records fixes to earlier mistakes.

To test the hypothesis, we compared a MAS against a single LLM with the same tool access and a single LLM with raw-data-only access, isolating the effects of orchestration and of tool access. We evaluated accuracy, token usage, and inference time on a ground-truth set of sixty Premier League queries: thirty data-analysis questions scored against hand-verified canonical models, and thirty prediction questions scored on Brier score and correct delegation to the prediction model. The ground truth was frozen via content-hashing to prevent benchmark drift between runs, so any change in results reflects the system under test rather than a moving target. Splitting the set evenly between deterministic and predictive questions was deliberate: the two workloads stress different parts of the pipeline (SQL generation and retrieval on one side, calibrated forecasting on the other), and a single blended score would have hidden where each configuration actually succeeds or fails.

Our first finding concerns autonomy. Limiting the MAS to predefined SQL templates yields high in-coverage accuracy but collapses on out-of-coverage questions, since templates cannot express computed columns, filtered aggregates, or cross-season logic. With Claude Opus 4.8, full autonomy recovers what templates cannot, reaching a production-grade 94% out-of-coverage accuracy. But autonomy is not free: it costs more API calls, and with other foundation models such as Gemini 3.1 Pro the self-debate loop failed repeatedly, dropping accuracy to 70%. Autonomy is more expressive but subtly wrong more often, and behaviour varies by model because training data differs; the same pipeline can therefore be production-ready under one foundation model and unreliable under another, which matters for any deployment that plans to swap providers on cost. This experiment used the original single-league substrate; the MAS-versus-single-LLM comparison was re-scoped to the multi-league database (see Limitations).

Second, structured database access, delivered as YAML schema, is the key driver of accuracy, while orchestration is useful but not critical. Giving a single LLM tool access versus raw-data-only access established 87% accuracy. This disagreed with our initial hypothesis: on the same questions, the MAS matched the single LLM exactly, both at 87% (26/30). A paired McNemar exact test found no significant difference ($p = 1.0$), indicating orchestration adds no measurable accuracy on this workload. The cost, however, differs sharply. The MAS averaged 32,324 tokens at 16.7 seconds, against 22,150 tokens and 8.1 seconds for the single LLM: roughly 1.5× the tokens and 2× the latency for no accuracy gain. Its value, if any, lies in engineering robustness rather than in measured answer accuracy: error isolation, inspectable control flow, and the ability to swap or debug one agent without disturbing the rest.

Third, memory's benefit depends on the setting. On single-run queries such as the golden set there is no benefit, because retrieval has nothing prior to draw on. The SQL cache nonetheless

delivers clear cost savings on recurring queries, roughly 44% fewer tokens and 17% lower latency on cache hits, while remaining accuracy-neutral. A dedicated session-based evaluation showed conversation memory has no net effect on reference-resolution accuracy against a matched no-memory baseline (+0.000), though it far outperforms a stateless single LLM with no history (+0.441). A full-history single LLM does better still, but at a substantially larger token cost that makes it impractical in production. The correction store adds a further safeguard: recorded fixes prevent the system from repeating an error it has already been corrected on within a session.

Prediction results demonstrate the MAS's ability to delegate to specialised tools rather than reasoning about outcomes in free text. The calibrated model improved accuracy 12.4 points over a base-rate baseline, measured by Brier score on a held-out 2024–2025 validation set of 379 fixtures. The system uses a stacking ensemble with XGBoost and LightGBM base learners, a logistic meta-learner, and isotonic calibration, so the probabilities it returns are calibrated rather than merely ranked. A key limitation is that argmax decisions prevent it from predicting draws, which make up roughly a quarter of fixtures and remain the hardest class for any football model.

Several limitations apply. The golden-set results come from a single run, so differences of one to three questions fall within noise, and the data-analysis benchmark is small relative to comparable agentic-systems papers. The autonomy and comparison experiments also ran on different database substrates, so those two findings should be read side by side rather than pooled. Even so, the implementation offers a clear, justifiable contribution for the industry partner and for practitioners in AI systems engineering.

We present a blueprint architecture for a football MAS covering knowledge-base design, degree of agent autonomy, pipeline and state management, memory management, and human feedback, alongside proprietary tools for analysis, visualisation, and prediction. The highest-leverage investment is clean, well-documented structured data access, since that is the main source of accuracy; a documented schema specification is more valuable than fancier orchestration. Multi-agent orchestration is worthwhile chiefly for the robustness it provides on ambiguous, previously unseen questions, where higher token and latency cost is justified; a single LLM with tools is adequate for predictable query loads. A memory layer is most beneficial for predictable multi-step conversational loads, amortising repeated work through caching and improving reference resolution across turns, provided extracted parameters match exactly, so results are never served from the wrong season. Predictions should be communicated as calibrated probability distributions rather than single most-likely outcomes. This project contributes to the literature on AI agents a controlled decomposition that separates the effects of tool access and orchestration, and, more practically, a set of engineering defaults that let a team decide when the extra cost of a multi-agent system is warranted and when a single tool-using LLM will do.

1 Introduction

Large language models have become the interface layer for data-driven analysis, letting users with little statistical background turn natural language into database queries that extract evidence and synthesise explanations Yu et al. (2018). Sports analytics is a natural fit: the domain is rich in structured data (match events, player statistics, league tables), yet a fan, journalist, and analyst all query the same underlying data regardless of their question Rein and Memmert (2016). A conversational agent that can answer "which team had the best goal difference last season?" or forecast an upcoming fixture lowers the barrier between raw football data and the people who reason about it.

1.1 Technological Ecosystem

A growing body of work, including AutoGen Wu et al. (2023) and MetaGPT Hong et al. (2024), argues that multi-agent systems (MAS) outperform single monolithic LLMs on complex, multi-step tasks. Decomposing a task across specialised agents, an orchestrator that routes intent, a data agent that generates and validates queries, a synthesiser that composes the answer, coordinated through an explicit control flow can outperform asking one model to plan, retrieve, compute, and explain in a single pass. The benefit does not stem from scale but from orchestration (each agent operates over a narrower context), tool delegation (computation is delegated to databases and models rather than hallucinated), and error isolation (a validation or evaluation agent repairs failed steps before they propagate), codified agentic design patterns Gulli (2025) we apply to football analytics.

1.2 Football Outcomes

Crucially, this MAS-versus-single-LLM comparison has not been tested on football data. Football analytics research is dominated by machine-learning models for specific predictive tasks: expected goals, match-outcome forecasting, player valuation, injury risk, each a bespoke model evaluated in isolation Decroos et al. (2019); Rossi et al. (2018). These do not address how best to organise an LLM-driven analytics system over football data. To our knowledge, no prior work offers a controlled architectural comparison, MAS against single-LLM baselines, on a football workload alongside a calibrated predictive component. Realising these benefits requires several established practices, which we adopt and evaluate. Retrieval-augmented generation and a domain knowledge base ground outputs in authoritative data rather than the model's parametric knowledge. The control-flow architecture, how branches are wired and failures retried, determines whether decomposition helps or merely adds latency. A memory layer (conversation memory, a query cache, a corrections store) lets the system carry context across turns and amortise repeated work. Finally, football's defining question, who will win, demands a genuine predictive model, so we integrate a calibrated match-outcome predictor as a first-class tool, benchmarked against market-implied probabilities and naive baselines.

1.3 Research Focus and Objectives

The main goal of this research project is to determine whether a multi-agent architecture is the correct choice for conversational football analytics and when it optimizes a system that provides answers that are grounded in reliable structured data. Drawing on publicly available data (Section 3), this Capstone Project develops methods to better understand AI and agentic architecture. Our research focuses on four main questions:

- **Autonomy:** How does the degree of autonomy granted to a data analyst agent, fully controlled, semi-controlled, or fully autonomous, impact the accuracy on both in-coverage and out-of-coverage queries?
- **Orchestration vs Tools:** Does multi-agent orchestration add accuracy beyond simple tool access, once tool access and schema context are held constant for a system grounded in football?
- **Memory:** What is the contribution of persistent memory to accuracy, latency, and token cost, both for isolated queries with no prior context and for multi-turn conversational sessions where earlier turns can be reused?
- **Prediction:** Can a calibrated match-outcome model, leveraged as a specialized agent tool, produce reliable probabilistic forecasts relative to market-implied and naive baselines?

1.4 Main Contributions (Results)

As described in the executive summary, we find that structured tool access is the leading determinant of accuracy: a single LLM moving from raw-data-only access to tool access improves from 70% to 87% overall. This gap is partly attributable to coverage rather than reasoning, since the raw-data condition cannot answer questions whose source tables are not inlined into its context. Multi-agent orchestration added on top of tool access yields no further measurable accuracy: the MAS and the single tool-using LLM both score 87% (26/30), a difference that is not statistically significant (McNemar exact, $p = 1.0$).

1.5 Report Structure

This work is organised as follows: Section 2 reviews related work on multi-agent LLM system and football prediction. Section 3 illustrates data sources and the knowledge-base architecture of this project. Section 4 sets out the system design and methodology for functional deployment, this includes the MAS architecture, memory layer, and prediction models. Section 5 details the experiment designs, covering the single-LLM baselines, the ground-truth dataset and evaluation process, and the three experiments on agent autonomy, orchestration versus tools, and memory. Section 6 reflects on our findings and reported results. Section 7 provides a discussion on limitations and areas to explore further research. Lastly, section 8 concludes with guidelines for end users and directions for future work.

2 Literature Review (Related Work)

2.1 Evaluation gaps in multi-agent LLM systems and orchestration design patterns

LLM-based multi-agent systems are well documented in theory. Li et al. (2024) survey the field around five components (profile, perception, self-action, mutual interaction, evolution); Yang et al. (2024) locate the advantages in task decomposition, specialisation, and proprietary-data preservation; Ye et al. (2025) train an LLM to generate executable MAS in a single inference; and Gulli (2025) systematises 21 agentic design patterns on LangChain/LangGraph, CrewAI, and Google ADK. Our Manager, Data Analyst, and Visualizer pipeline, bounded retry loop, and rule-based Eval agent instantiate these patterns. The common limitation is evaluation: this evidence base is general-purpose QA and reasoning benchmarks that do not isolate orchestration from tool and database access, nor report its token-and-latency cost. Our MAS-vs-single-LLM-with-tools-vs-single-LLM-on-raw-data design decomposes exactly that effect, and finds that tool access, not orchestration, drives accuracy.

2.2 Multimodal vs. structured-query approach in agentic systems for football

The closest domain effort is SoccerAgent (Rao et al., 2025), which contributes SoccerWiki (a multimodal knowledge base), SoccerBench (a ∼10K multimodal QA benchmark across 13 tasks), and a MAS that decomposes soccer questions and outperforms MLLM baselines. Its focus is multimodal understanding, whereas we target structured analytical querying plus a calibrated predictive tool. It also benchmarks only against MLLMs, omitting the controlled MAS-vs-single-LLM-with-equal-tools ablation, the autonomy sweep (controlled, semi-controlled, autonomous), and the memory experiment that form our empirical contribution.

2.3 Calibration and Evaluation practices in match-outcome prediction

On prediction, prior work is bespoke single-model research, not an agent tool. Constantinou (2019) combines dynamic ratings and Hybrid Bayesian Networks across 52 leagues. Wong et al. (2024), are the closest analogue to our predictor, combining standard classifiers, LightGBM, and a CNN through stacking and voting ensembles on the EPL, benchmarked against bookmaker odds. Bunker and Thabtah (2019) stress chronological train/test splitting and warn against accuracy-only evaluation, and Wheatcroft and Sienkiewicz (2021) argue for calibration-sensitive scoring, together motivating our pre-kickoff features, out-of-sample testing, and use of Brier score, log loss, and isotonic calibration. Iskandaryan et al. (2020) and Rossi et al. (2018) add weather and GPS-injury modelling respectively. Across all of these,

each predictor is evaluated in isolation against odds or naive baselines, never exposed as a callable tool inside a conversational system. We embed a calibrated stacking ensemble as a first-class agent tool and evaluate the whole system under the controlled comparison this literature has reserved for general-domain benchmarks.

2.4 Memory and Retrieval in Conversational Agents

Memory is a core agentic design pattern established by Antonio Gulli, a senior engineer at Google (Gulli, 2025). He describes designing a process that manages memory by distinguishing between its short-term version which is the conversational context within a single session, from long-term memory which is a persistent and browsable layer that an agent queries via retrieval augmented generation (RAG). This process is proven to be useful because it allows the system to recall facts, preferences or prior interactions across different tasks. This dual structure is the foundation of this project's conversation-memory, SQL-cache, and corrections-store design, which we will go more into detail in the following sections. The literature motivates evaluating each registry separately instead of doing so in a unified way. Additionally, computer scientist and author of the book AI Engineering, Chip Huyen, wrote about the differences in exact caching, which reuses findings whenever an incoming query's embedding falls under a similarity threshold of a query that has already been cached. She suggests the problematic nature of semantic similarity that can take place with caching, because this requires high-quality embeddings for the similarity metric to be consistent making it more failure-prone otherwise. If this is not properly configured, then wrong cached answers for specific queries may be inappropriately retrieved.

3 Data

3.1 Data Sources

Our football analytics knowledge base integrates complementary providers whose records differ in granularity, identifiers, access method, and update frequency. The club-football component covers twelve seasons, from 2014–15 through 2025–26, across ten European top-flight competitions: the English Premier League, Spanish La Liga, German Bundesliga, Italian Serie A, French Ligue 1, Dutch Eredivisie, Portuguese Primeira Liga, Belgian Pro League, Turkish Süper Lig, and Scottish Premiership. The sources jointly provide match events, player and team performance, line-ups, expected-goal metrics, market values, transfers, injuries, and pre-match market information. The database scope is broader than any individual analytical task. Different capabilities require evidence at match, event, player-match, player-season, team-season, transfer, injury, or market-value level. The effective sample for an analysis therefore depends on the level at which its records are defined and on its eligibility conditions, not on the overall size of the database. The integrated data from all providers is consolidated into a single managed PostgreSQL instance hosted on Aiven, giving every agent one authoritative source to query at runtime rather than reconciling providers on demand.

WhoScored contributes event-level records for passes, shots, tackles, dribbles, fouls, duels, set pieces, and goalkeeper actions. Each event is linked to a match, team, and, where applicable, player, and includes timing, pitch coordinates, outcome information, and provider-specific qualifiers. The pipeline decomposes these nested qualifiers into directly queryable boolean, numeric, and categorical fields so that downstream agents do not need to parse semi-structured strings at query time. The WhoScored `game_id` is retained as an important provider key for match-level reconciliation.

Understat supplies attacking and possession-chain information at player-season, player-match, and shot level. Its variables include goals, assists, minutes, shots, expected goals (xG), expected assists (xA), non-penalty expected goals, `xGChain`, and `xGBuildup`, together with contextual splits by position, match situation, shot zone, and shot type. These measures complement observed goals by representing the quality and development of attacking opportunities.

FBref is collected at both match and season level. Match reports provide line-ups, formations, substitutions, referee and venue metadata, and separate outfield-player and goalkeeper performance tables. Season-level pages contribute league standings, home-away splits, squad aggregates, opponent aggregates, attendance, and top-scorer information. These records connect individual match performance to wider team and season context.

Transfermarkt contributes player profiles, historical market-value snapshots, transfer records, and injury spells. These tables provide age, nationality, position, preferred foot, transfer origin and destination, transfer fee, injury dates, and days or games missed. Every time-varying record is aligned to its observation date so that analytical frames use only information available before the relevant reference date.

Football-Data supplies match-level validation statistics and pre-match bookmaker odds. Its variables provide an additional source for selected match statistics such as scores, shots, corners, and cards. When market information is used, decimal odds are converted to implied probabilities and normalised across the home, draw, and away outcomes to remove the bookmaker overround.

A historical FIFA World Cup dataset obtained through Kaggle contributes a limited international-tournament supplement. It is processed through the same staging, cleaning, validation, and naming-standardisation framework as the club data. Its role is deliberately secondary: it expands query coverage for international tournaments without determining the structure of the club-focused knowledge base.

Figure 3.1 summarises the complementary role of the providers. A primary contribution indicates that a source supplies the main representation of an information family, while a supporting contribution adds complementary variables or validation evidence.

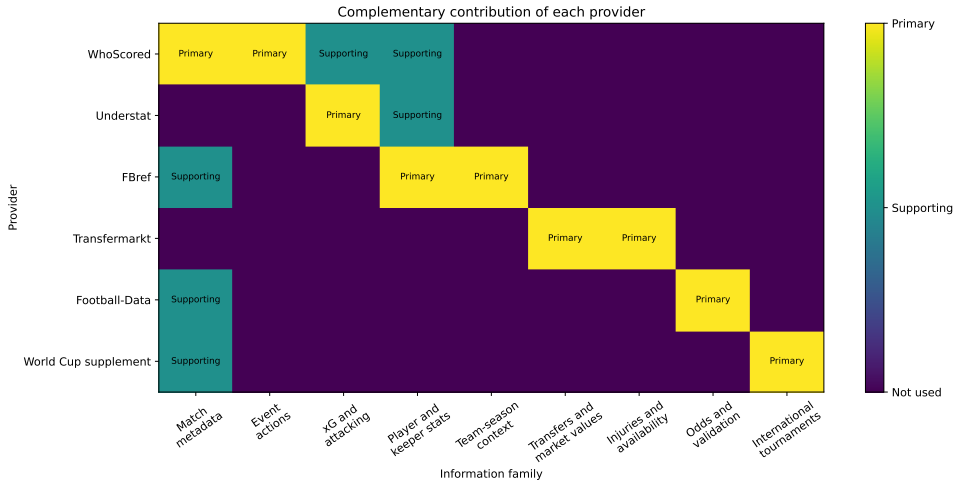


Figure 3.1: Primary and supporting contributions of each provider to the integrated knowledge base. Empty cells indicate that the provider is not used for the corresponding information family.

3.2 Data Engineering Pipeline

The central engineering challenge is not the volume of any individual source, but the integration of records created under incompatible schemas, identifiers, naming conventions, access methods, and observational grains. We therefore use a reproducible ETL (extract, transform, and load) pipeline with five stages: raw extraction and staging, source-specific transformation, entity resolution, validation and loading, and construction of agent-facing serving datasets.

Figure 3.2 summarises the system architecture. Additional competitions, seasons, or records change the contents of the database but do not change the role of the layers.

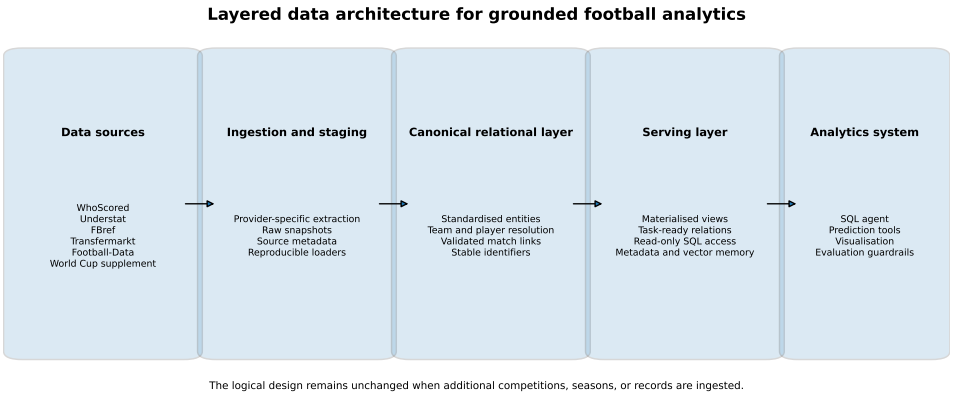


Figure 3.2: Layered data architecture for grounded football analytics. Raw provider records are progressively transformed into canonical entities, task-ready serving relations, and evidence accessible to the analytical system.

3.2.1 Extraction and Raw Staging

The pipeline combines programmatic scraping, provider-specific Python libraries that query data APIs directly, direct file downloads, and static external datasets. WhoScored, Understat, FBref, Transfermarkt, Football-Data, and the World Cup supplement are collected separately because they differ in access method, schema, update frequency, and granularity. Each source is first written unchanged to a raw staging directory. Original column names, provider identifiers, the provider's original hierarchical (nested) data structures, and source files are preserved alongside provenance metadata recording the source, competition, season or tournament edition, extraction date, and pipeline version. This immutable layer allows failed transformations to be reproduced without repeating upstream collection and provides an audit trail when a provider changes its schema.

3.2.2 Source-Specific Cleaning and Transformation

Source-specific transformation jobs convert the raw inputs into standardised intermediate frames. Column names are mapped to a common convention; dates and kick-off times are parsed into explicit temporal types; competition and season labels are mapped to canonical codes; formatted numeric and monetary fields are cleaned and cast; and duplicated records

are removed using keys appropriate to each table's grain. WhoScored's nested event qualifiers are decomposed into explicit boolean, numeric, and categorical fields. FBref's multi-level HTML headers are flattened before match metadata, team statistics, line-ups, player statistics, and goalkeeper statistics are separated. Understat data are reshaped according to their shot, player-match, player-season, and contextual split grains. Transfermarkt market values, transfers, and injury intervals are standardised and aligned to canonical players and dates. Football-Data odds are transformed into normalised implied probabilities. The World Cup data enter the same transformation framework. Column names and data types are standardised, country names and tournament editions are normalised, and duplicated records are removed using the source identifiers or the relevant match-level keys.

3.2.3 Entity Resolution and Cross-Source Integration

After source-level cleaning, records are mapped to canonical competition, season or tournament, team, player, and match entities. Provider-specific identifiers remain available for traceability, while downstream fact tables use stable internal identifiers. Team mappings are resolved first through deterministic alias dictionaries. These mappings handle abbreviations, punctuation differences, sponsorship names, and known historical variants. Player mappings combine normalised names, transliteration, whitespace and punctuation cleaning, and fuzzy comparison. Candidate matches are cross-validated using birth year, nationality, club, season, and team context when those attributes are available. Matches are reconciled using provider identifiers where possible and otherwise through competition, season, date, home team, and away team. Ambiguous mappings are logged for manual review rather than silently accepted. Players who appear in event records before a complete profile is available are inserted as stub entities, preserving referential integrity and allowing later enrichment.

3.2.4 Validation and Database Loading

Before loading, every transformed frame is validated against an explicit schema contract. The checks cover required columns and data types, primary-key and composite-key uniqueness, foreign-key coverage, permitted competition and season values, and plausible numeric ranges. Match-level checks verify that the home and away teams differ, that the match date is compatible with the season, and that the recorded result is internally consistent with the final score. Child tables such as events and player-match performance are checked against their parent matches. Player-season records are checked against the relevant player, team, season, and competition dimensions. Transfer, market-value, and injury observations are checked for valid dates and player references. Validated frames are loaded into PostgreSQL in dependency order: competitions, seasons, teams and players, matches, and finally event, player-performance, team-season, transfer, injury, and market-value tables. Idempotent upserts and explicit conflict keys allow the loader to be rerun without creating duplicate records.

3.2.5 Serving Tables and Model-Ready Features

The serving layer separates general analytical access from task-specific feature construction. Materialised views pre-compute frequent joins and aggregations for matches, player matches, player seasons, shooting profiles, availability, and team seasons. These views reduce query complexity and limit the amount of high-volume event data scanned by agent-generated SQL. When task-specific features are constructed, all time-varying information is aligned to the relevant analytical date. Rolling match variables are ordered chronologically and shifted so that the current match cannot influence its own features. Market values, transfers, injuries, recent form, rest intervals, and availability measures use only records observed on or before the target date. This preserves temporal integrity and prevents information leakage.

3.3 Knowledge Base

The knowledge base is implemented as a three-layer relational architecture on managed PostgreSQL. The raw layer preserves provider-native records for audit and reprocessing. The canonical layer stores normalised competitions, seasons, teams, players, matches, and associated fact tables with unified identifiers. The serving layer exposes stable analytical views and pre-computed summaries to the agents. The principal dimensions are competitions, seasons, teams, players, and matches. The fact tables preserve distinct levels of granularity:

- one row per match in the match table;
- one row per event in the event table;
- one row per player and match in the player-match table;
- one row per player, team, and season in the player-season table;
- one row per team and season in the team-season tables;
- one row per dated market-value observation;
- one row per transfer;
- one row per injury spell.

Preserving these grains prevents accidental duplication during joins. For example, joining a match directly to event records multiplies the match row by the number of events, while joining a player-season record to transfers can create several rows for one player-season. Task-specific queries therefore aggregate or filter each fact table before combining grains. The highest-volume event and player-match relations are partitioned by competition and, where appropriate, by season. Partition pruning limits scans to the relevant subset. Indexes support common access paths involving player, game, season, competition, team, and date identifiers. Materialised views expose stable, semantically meaningful columns to the analytical system. The principal serving relations are summarised in Table 3.1.

Table 3.1: Principal serving views in the knowledge base.

Serving relation	Purpose
`vw_match_summary`	Match metadata, canonical team names, scores, results, formations, match statistics, and market odds.
`vw_player_match_summary`	Player-match performance linked to canonical player, match, team, competition, and season context.
`vw_player_season_summary`	Player-season production, profile attributes, contextual statistics, and eligible market-value information.
`vw_player_shooting_profile`	Shot-level locations, outcomes, body part, situation, direction, assistance, and related contextual variables.
`vw_player_availability`	Canonical injury and availability history with dates, matchweeks, days missed, and games missed.
`vw_team_season_summary`	Standings, home-away splits, squad statistics, opponent statistics, goalkeeping measures, and season-level team context.

The serving layer allows an agent to query a documented match or player summary without reconstructing the underlying joins between dimensions and provider-specific fact tables. The agents receive read-only database access, and generated statements are checked before execution. Queries that attempt to modify data or use unsupported relations are rejected.

3.4 Exploratory Data Analysis

The exploratory analysis is conducted after cleaning, entity resolution, and validation. Its purpose is to characterise structural properties relevant to downstream analytical tasks rather than to display the temporary scale of the operational database. The paper therefore uses a frozen analytical extract and reports normalised distributions and aggregate feature availability. The figures avoid raw row totals, league-specific coverage matrices, and season-by-season counts. This keeps the section stable as the operational database expands while preserving a reproducible empirical extract for the paper.

3.4.1 Match-Outcome Composition

A football match has three mutually exclusive full-time outcomes: home win, draw, and away win. Figure 3.3 reports their relative prevalence in the frozen analytical extract. The class distribution is relevant because raw predictive accuracy can be misleading when one outcome is more common than the others. Any future match-outcome capability should therefore be compared with an empirical base-rate benchmark and assessed with class-specific and probability-sensitive diagnostics rather than accuracy alone.

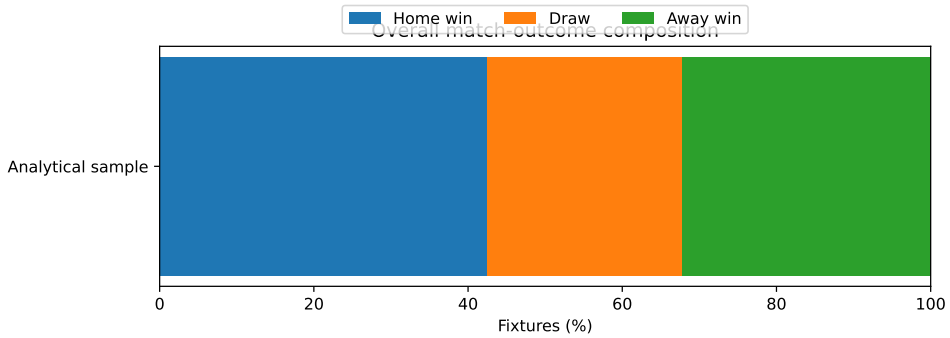


Figure 3.3: Overall composition of home wins, draws, and away wins in the frozen analytical extract. The figure reports relative shares rather than absolute match counts.

3.4.2 Goal-Scoring Profile

The total number of goals in a match provides a compact description of the scoring environment represented by the data. It is calculated as the sum of the final home and away scores. Figure 3.4 reports the normalised distribution of total goals per match. Matches with five or more goals are grouped into a common upper category so that rare high-scoring fixtures do not dominate the scale. The goal distribution provides context for later analytical tasks. Football

is generally a low-scoring process, so a small number of actions can materially change the final result. This makes uncertainty, team strength, recent form, shot quality, and contextual variables important for any future outcome analysis.

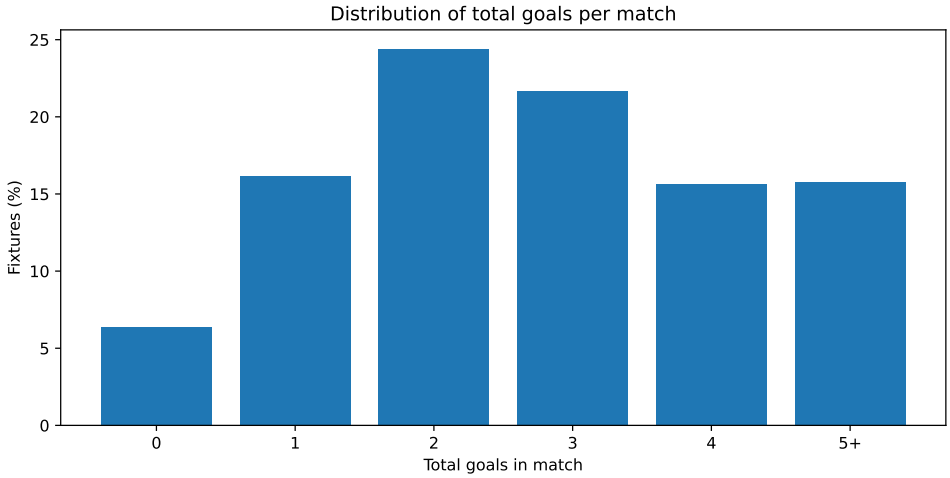


Figure 3.4: Normalised distribution of total goals per match in the frozen analytical extract. Matches with five or more total goals are grouped in the rightmost category.

3.4.3 Feature Availability

The integration of several providers creates a broad match-level feature space, but the same variables are not necessarily observed for every record. Historical reporting conventions, source availability, and extraction differences can create missing values even when the match itself is present. Feature availability is evaluated by coherent information family rather than by individual league, season, or database column. The families include final results, half-time results, formations, shots, shots on target, corners, fouls, cards, and market odds. A match is considered complete for a family only when every component field assigned to that family is non-missing.

Figure 3.5 reports the aggregate completeness rate for each family. This identifies which groups of covariates support broad analytical coverage and which require explicit eligibility filters or missing-data rules.

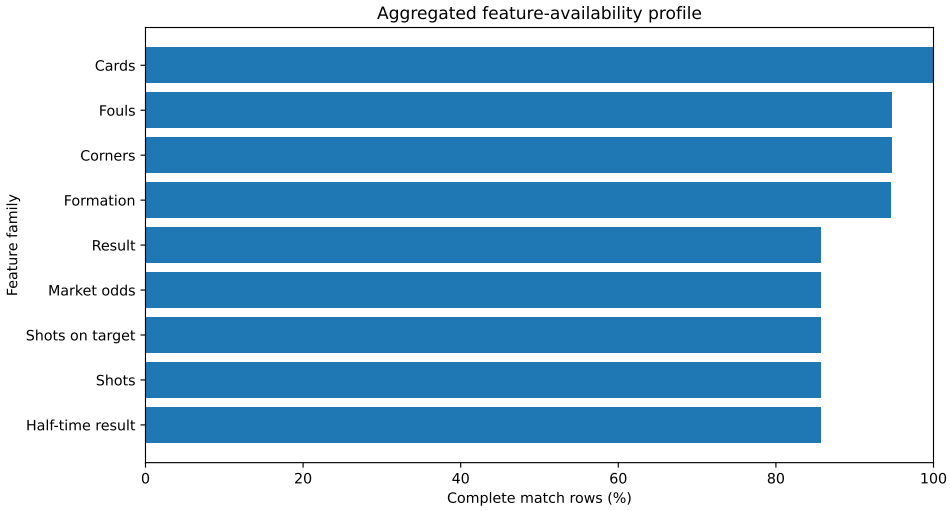


Figure 3.5: Aggregated feature-availability profile by information family in the frozen analytical extract. A match is complete for a family only when all fields assigned to that family are non-missing.

3.4.4 Implications for Downstream Analytics

The exploratory findings have four direct implications for downstream analysis. First, the three-class match-outcome structure should be preserved. Draws should not be removed simply because they are more difficult to distinguish, and performance should not be interpreted through accuracy alone. Second, the goal distribution reinforces the need to represent uncertainty. Relatively low scoring and small final-score differences imply that analytical tools should preserve probability estimates and contextual evidence rather than returning only deterministic labels. Third, feature availability must be incorporated into tool eligibility. Capabilities based on broadly available variables can be offered across the knowledge base, while tasks requiring less consistently observed information should declare their required fields and reject unsupported requests rather than silently fabricating or imputing unavailable ev-

idence. Fourth, all time-varying variables must be aligned to the analytical date. Market values, transfers, injuries, team form, and rolling statistics must be constructed using only observations that precede the target event.

3.5 Schema Documentation: YAML Specification

The relational schema is accompanied by a machine-readable YAML specification used as context for the language-model components. The specification documents each relation's purpose, observational grain, primary and foreign keys, important columns, permitted joins, temporal meaning, and known caveats. This metadata layer serves two functions. First, it improves SQL generation by providing a semantically explicit description of the database rather than only a list of table and column names. Second, it supports evaluation by making the expected grain and valid join paths available to the rule-based checks. The YAML documentation is maintained alongside the SQL schema. Any structural change to a table, materialised view, key, or column must therefore be reflected in both representations. This prevents the language model from relying on stale schema context and preserves consistency between the physical database and the interface presented to the agents.

4 Methodology

4.1 Multi-Agent System Architecture

We orchestrate the agents with LangGraph, which represents the pipeline as a directed graph (Figure 4.1): nodes are agents, edges define control flow, and a shared state object is passed between them, making the pipeline explicit and inspectable. Conditional routing, for example, forecast versus retrospective queries, is expressed as edges out of the Manager.

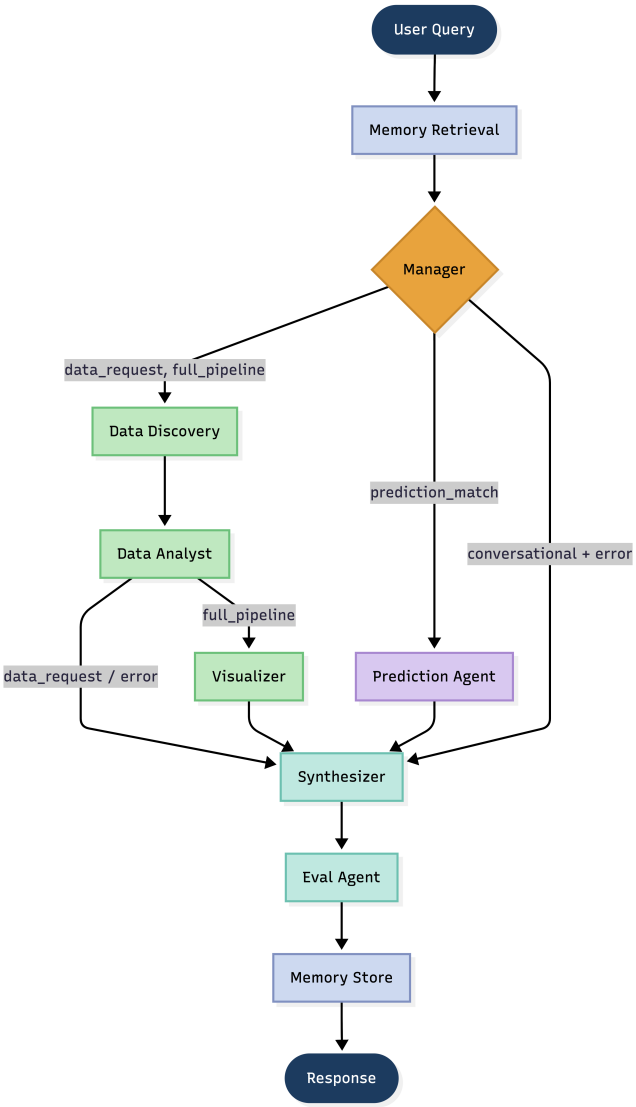


Figure 4.1: The LangGraph orchestration graph. The diamond denotes the Manager's routing decision; edge labels give the intent value(s) governing each branch. Data Analyst has a second, internal routing split (`full_pipeline` vs. `data_request`/error) determining whether Visualizer is invoked before Synthesizer.

Manager and Routing. The Manager Agent decomposes the query intent, plans which agents to involve in the execution, and extracts structured parameters (teams, seasons, metrics). Deterministic queries route to the data-analysis branch, forecasts to the prediction agent, and out-of-scope inputs directly to the synthesiser. This gives a well-defined per-query path.

Shared preamble (all agents).

```
You are a Premier League data analyst. Answer the user's question using
ONLY the database. Rules: seasons are 'YYYY-YYYY' (hyphen, e.g.
'2023-2024'); the latest season is 2024-2025 (no 2025-26 data); point
deductions are already in the points columns; prefer IDs over names for
joins; canonical team names (e.g. 'Manchester United', 'Wolves').
```

MAS Manager (intent + entity extraction).

```
Classify and extract for this Premier League question.
Question: {question}
Respond JSON: {"intent": "data_request"|"prediction_match"|"conversational",
"seasons": ["YYYY-YYYY", ...], "teams": [...], "metrics": [...],
"home_team": "...", "away_team": "...", "match_date": "YYYY-MM-DD"}.
Use intent='prediction_match' ONLY when asked to forecast a specific
fixture's outcome; then fill home_team, away_team and match_date.
Resolve 'this/current season' to 2024-2025 (latest data).
```

A Data Discovery agent identifies the relevant schema tables and columns; the Data Analyst agent then generates two SQL queries, and if the two queries diverge, the agent reconciles them to find an improved query.

MAS Data Analyst, AUTONOMOUS (free SQL + self-debate). Two independent candidates are generated per iteration (up to 2 reconcile rounds); their result sets must agree or the answer is flagged low-confidence.

```
# Candidate A
{preamble}
Database schema:
{schema}
Question: {question}
Write the most direct correct query.
Return ONLY one SQL SELECT statement, no prose, no code fences.

# Candidate B
... Write an independent query (different phrasing/joins) for the same answer.

# Reconcile round (if A and B disagree)
```

```
... Your two previous SQL attempts DISAGREED on their results -- reconcile them.
Attempt A: {sql_a}
Attempt B: {sql_b}
Find the correct query.
```

The Synthesiser composes a natural-language answer; a rule-based Eval agent then checks it against the retrieved figures, adding no model calls. Inconsistencies are flagged and, on a detected error, written to the corrections store for future reuse.

4.2 Memory Architecture

Before breaking down the functional aspects of memory, it is imperative to understand what it actually entails, as it sounds like a human trait that is usually foreign to machines. However, in this context, it refers to the capacity for an algorithmic system to encode, store, retain, and retrieve details from data or past actions, capacities that are foundational to what we colloquially call machine "intelligence." As outlined in their paper, *Memory in the Age of AI Agents: A Survey*, Hu et al. (2026) determine three roles that memory can serve for an agent: what the agent actually knows, a declarative record of facts and prior interactions; how the agent improves, procedural knowledge distilled from past trajectories, including failures and successes; and working memory, which exists within a session and can be thought of as what the agent is thinking about in the present moment. Hence, we adopt a similar framework by applying a conversation memory store that instantiates factual memory to maintain cross-session consistency. SQL cache and correction memory serve the purpose of case-based experiential first-hand memory based on prior runs that get stored for reuse: successful SQL gets stored for reuse, whereas the correction store is used to evaluate failed patterns as a corrective warning for future use cases. Finally, the knowledge base stores documents and accommodates retrieval-augmented generation (RAG), following the aforementioned paper's distinct process.

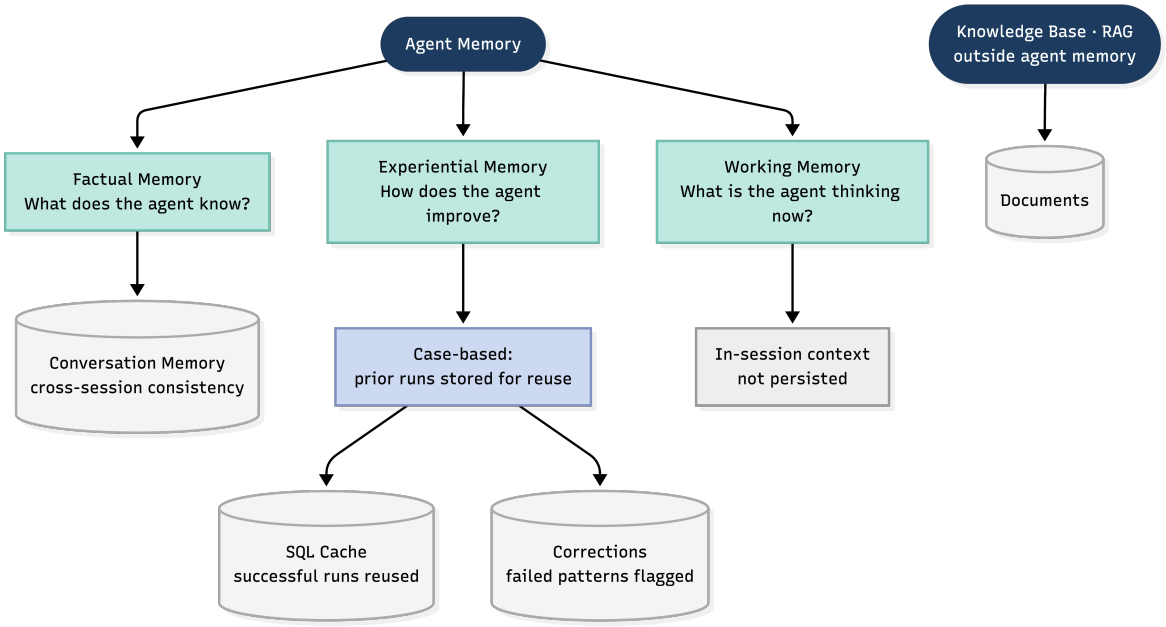


Figure 4.2: Functional taxonomy of agent memory, following Hu et al. (2026), mapped onto MA-FDSS's memory stores. `documents` is drawn as a separate root rather than a child of Agent Memory, reflecting its classification as retrieval-augmented generation over a static knowledge base rather than agent memory proper.

Our MAS architecture is stateful across turns thanks to the memory layer built on PostgreSQL. This means that the system remembers context and past conversations. This was built using PostgreSQL's `pgvector` extension, hosted on Aiven. Memory is decomposed into four `pgvector` tables, outlined in Table 4.1. They all share the same Aiven-hosted store, embedding function, and retrieval node, simply put, the actual database, a model that turns text into vectors, and the LangGraph node that queries the tables during a turn. Three of the stores are question-keyed, meaning that the actual question or prompt a user asks gets stored as an embedding, and any new query is matched against previously asked questions rather than the underlying data. This context gets injected live as the request graph executes. The fourth is content-keyed: because it holds documentation, the query is matched directly against the source text itself rather than against a prior question. This store is populated through an offline ingestion pipeline, for example, ingesting player profiles and match reports that provide qualitative context to a question. Note that the agents don't update this store, because it is static and the data is purely contextual. Even though it is not considered agent memory in the traditional sense, it is still part of the agent's static RAG knowledge access. There are two auxiliary tables, `feedback` and `trajectory_log`, that record signals for offline analysis, which the agents don't access during a turn.

Table 4.1: The four `pgvector` tables. Similarity is cosine on 384-dimensional embeddings; thresholds are applied after retrieval.

Store	Keyed on	Written when	Threshold
`conversation_memory`	user question	turn succeeds	0.70
`sql_cache`	user question	SQL returns rows	0.88
`correction_memory`	user question	turn errors / is flagged	0.75
`documents` (RAG)	document text	offline ingestion	0.30

Embedding model. The four database stores share a single embedding function: the `all-MiniLM-L6-v2` sentence encoder (384 dimensions), which is executed locally. Within `pgvector`, these embeddings are compared using a cosine distance operator. These get stored using an HNSW index in the tables in the database for approximate nearest-neighbor search. Since all stores use one encoder, the vector space stays consistent, so a question embedding can be reused for every question-keyed lookup within a turn.

Retrieval node. A dedicated memory retrieval node is utilized as the graph's entry point. It runs prior to the Manager classifying the intent of a prompt. In fact, all query types trigger a lookup against the conversation, correction, sql, and documents stores. Prior turns that are similar then get supplied to the Manager and Synthesizer agents as context. For example, a stored turn establishing that Arsenal finished the 2023–24 season with 89 points is resurfaced when a following query asks how that season compares to the one before. Separately, a cache hit sidesteps SQL generation downstream: a rephrased question such as "who scored the most goals last season" reuses the SQL already generated for a similarly phrased prior query, saving runtime and token spend. A correction-memory hit, meanwhile, surfaces as a cautionary warning, for instance, flagging a prior confusion between points and goal difference so the Data Analyst is less likely to repeat the same mistake. A documents-store hit, similarly, provides narrative context for qualitative answers.

Working memory. This refers to the LangGraph state object that is constructed fresh for each turn, and populated with the incoming query, retrieved memory context, intermediate tool outputs, and SQL results as the graph executes. Note that this gets discarded once a response is returned, as it does not have a dedicated store in the Aiven database, which was a deliberate engineering decision. For example, a question about Liverpool's clean sheets in 2024–25 carries the parsed team, season, and metric through the Manager, Data Analyst, and Synthesizer as transient state. However, this reasoning does not get stored to disk. There is also a session-level conversation-history variable that persists across a chat session but is write-only from the agents' perspective, accumulating as a running transcript that is never read back into any graph node, so it does not inform their decisions and resets when a new chat begins. This is the standard for most AI chat bots when a brand new conversation is started.

SQL-cache reuse guard. This feature dictates whether a previously generated query gets recycled and reused with no underlying changes. This makes false positives expensive, because a structurally identical question that differs in only a single data point, such as a team, season, or metric, can be served the wrong SQL. For example, if someone asks for the total number of goals in a specific season and then asks about a different season, the system could return the wrong values. Therefore, to solve this potential pitfall, we set up a two-stage gate. First, the nearest cached question must exceed a high cosine threshold of 0.88. Second, a consistency guard compares discriminating tokens between the new query and the cached item, to be as critical as possible before outputting a new answer. If there are any discrepancies during this process, the cache hit gets rejected. This prevents stale or entity-mismatched results from being displayed and solves any underlying issues before they arise.

Correction path. A single terminal Memory Store node runs after the Synthesizer and Eval Agent. Successful runs are persisted to conversation memory, and the SQL is cached when the query returns rows. Failures are recorded in correction memory, which is written to by two sources. The first is the Eval Agent, which executes a rule-based consistency check comparing the output against the retrieved data; it is designed to detect hallucinations made by the agents and writes a correction when one is found, for example a response claiming a player scored 20 goals when the retrieved data shows only 15 for that season. The second is a prediction-outcome hook, which writes a correction when a forecast's realised outcome is later supplied and differs from the prediction, such as a predicted home win that resolves as a draw. A user's response is not sent until all three writes have completed or failed silently, making this workflow due diligent. A storage failure will not break the turn, because each write is wrapped so that its failure is caught rather than propagated. This safety is independent of the fact that execution is synchronous on the response path: each write runs one by one, completing before the next begins in a chain, which adds some latency rather than protecting against it.

4.3 Services Used

We built our multi-agent system from open-source frameworks, favouring packages with strong adoption in the developer community. Agent orchestration is handled by LangGraph, which defines the agent graph, manages routing between nodes, and propagates shared state across the pipeline. To route requests across multiple frontier models through a single interface, we use LiteLLM, which normalises each provider's request and response format (including function-calling) into one provider-agnostic interface and lets us swap models via configuration rather than code changes. Models are served by the commercial APIs of OpenAI, Anthropic, and Google: GPT-5.5, Claude Opus 4.8, and Gemini 3.1 Pro Preview.

For observability we instrument the system with Langfuse, attached as a LiteLLM callback and exporting traces over OpenTelemetry; this provides the per-run token-usage and latency measurements reported in this paper. Our analytical data resides in a PostgreSQL database hosted on Aiven, which the Data Analyst agent queries through SQLAlchemy. The memory

layer reuses the same Aiven Postgres instance via the `pgvector` extension, storing conversation history, a cache of successful SQL queries, and a record of past corrections (Section 4.2); embeddings are generated locally with the `all-MiniLM-L6-v2` sentence-transformer model. The conversational interface is served with Chainlit, which provides a built-in chat front-end, and the application is deployed on Modal for serverless hosting.

4.4 Tools

The agents access three purpose-built tools, each exposed as a callable function rather than as direct access to any backend. The `use_template` tool offers a catalogue of predefined, parameterised SQL queries covering the most common analytical questions, and is the primary tool available to the controlled and semi-controlled Data Analyst variants. The `run_sql` tool lets the autonomous variant (and the semi-controlled variant as a fallback) issue free-form SQL against the football database, guarded by a read-only validation layer that rejects any non-`SELECT` statement. The `predict_match_outcome` tool wraps our trained match-outcome model: given a fixture (home team, away team, and date) it returns calibrated home-win, draw, and away-win probabilities, computed only from data available before kickoff to preserve a causal guarantee.

4.5 Prediction Node

As described in the Manager and Routing paragraph (Section 4.1), when the Manager classifies a query as a match-outcome forecast (or other prediction-related query), the prediction package gets activated. The agent node resolves the two teams, requests a forecast, and writes the result into the same state field used by the data-analysis branch. The prediction package is decoupled from the agent layer: it exposes a single forecasting interface and has no knowledge of LangGraph, the shared state, or the other agents, so it can be developed, trained, and evaluated independently of the multi-agent system. The agent node is the only point of contact, translating between the graph's state and the package's interface. Because the forecast is written into the same field the Data Analyst would populate, the Synthesizer and Eval agents downstream treat it identically to a data-analysis result, with no prediction-specific handling.

Rather than a sub-graph of discrete nodes for each capability, the MAS handles prediction through a single specialised LangGraph node with an internal Python router, a design forced by a DataFrame serialisation constraint. A further limitation of embedding a prediction agent within the MAS is that training and running a model on the fly is not time or cost efficient. Therefore, forecasting capabilities are restricted to pre-trained model artifacts, loaded at inference time based on the specific query.

This allows the MAS to fix the granularity of each query and identify a target type and feature builder before extracting the relevant pre-trained model artefact. Thirteen forecasting capabilities are currently available at inference, alongside an explanation layer appended on request.

Each of the thirteen capabilities belongs to one of six method families, defined in Table 4.2. Specifically: a calibrated ensemble for match outcome; uncalibrated parametric estimators for team-season and player-level targets; closed-form composite indices for deterministic targets; and, for the remaining tasks, counterfactual/ similarity-based reuse of a trained model or empirical-frequency estimates.

There are two skills that allow prediction agents to reuse a trained model, either counterfactually or via embedding similarity, so that they do not need to fit a dedicated estimator on repeated queries. The main prediction capability the MAS demonstrates, and the one fully detailed here, is the match-outcome model; its empirical results are reported in Section 6.

Table 4.2: Prediction capability families.

Family	**Method**	**Representative tasks**
Match outcome	Stacked ensemble (LR + XGBoost + LightGBM) + isotonic calibration	Match outcome
Team-season parametric	Multinomial logistic regression / gradient-boosted regression over team-season features	Season classification, position movement, points turnaround
Player parametric	Gradient-boosted classifier/regressor over player-season or cross-league features	Player decline, cross-league translation
Deterministic index	Closed-form composite (percentile/z-score blends, regression-slope weighting)	Home/away dependency, squad availability, stakeholder value, player value
Counterfactual/ similarity	Perturbs or embeds inputs to a trained model rather than fitting a new one	Player team-impact, transfer fit
Empirical frequency	Conditional outcome frequency vs. base rate	In-game event impact

Selecting the desired capability is a two-stage process. First, the manager agent sends the query to the LLM together with a fixed prompt listing the full set of task identifiers, and parses the LLM's free-text response into an intent label; for prediction goals, a task identifier is selected alongside the extracted model parameters (e.g., the two teams and match date). Subsequently, the prediction node resolves one of the thirteen prediction skills via a priority cascade (Figure 4.3). This is a three-step fallback sequence rather than a single lookup. Initially, it verifies the manager's task identifier, as described above. Then, the dispatcher falls back to a keyword check, for example, team names paired with matchup language like "relegation," matched directly against the user's raw query text. Note that the dispatcher falls back to a fixed default if neither the manager's label nor the keyword match succeeds in resolving a capability.

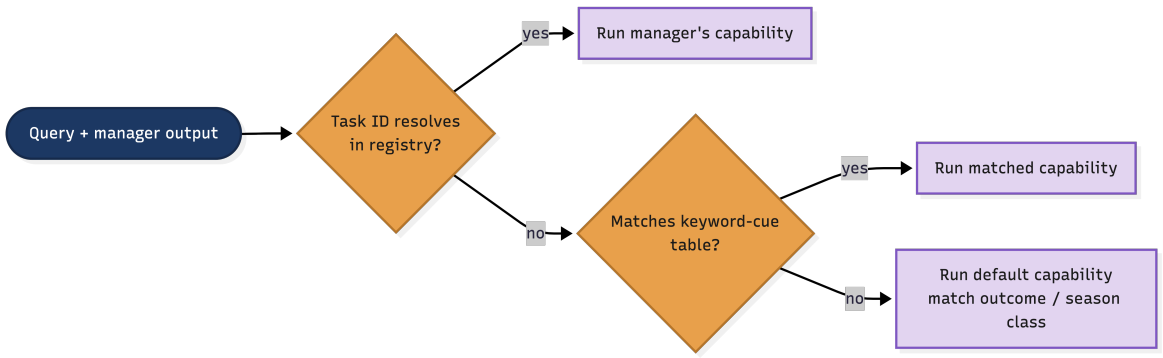


Figure 4.3: Capability selection cascade: the three-step fallback sequence resolving which prediction capability runs for a given query.

The layered nature of this design serves a distinct purpose. It keeps the routing decision interpretable at every step since there is always a traceable reason why a specific capability fired rather than an alternative single model decision. Additionally, the system becomes more robust to the distinct weaknesses of the manager step, which involves parsing a free-text LLM response by dividing strings. This keeps selection interpretable and degrades gracefully when the manager's free-text output is malformed. Figure 4.4 illustrates the prediction agent within the broader MAS pipeline.

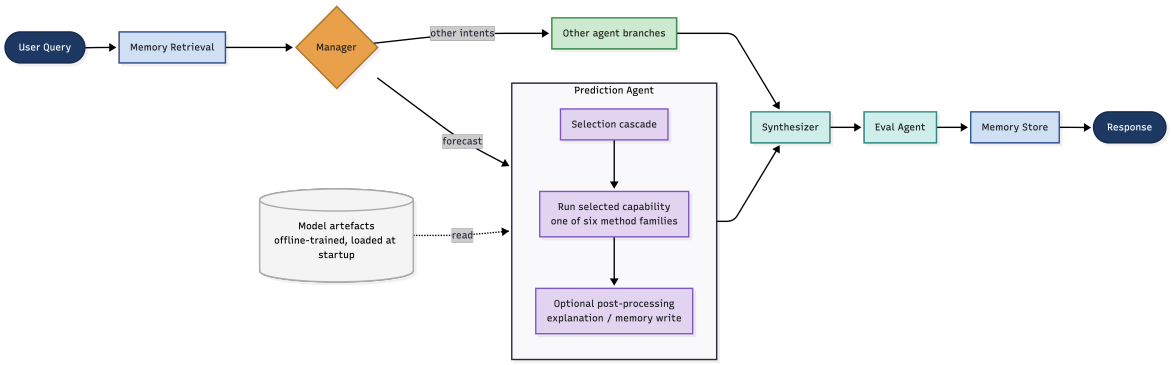


Figure 4.4: The prediction agent within the MAS pipeline: memory retrieval, manager intent classification, and the prediction-specific dispatch, execution, and post-processing stages, drawing on offline-trained model artefacts.

Of the six families in Table 4.2, match outcome prediction is the only one whose probabilities are explicitly calibrated, using isotonic regression. It is also the only family we evaluate against external baselines, namely a base rate predictor and market implied probabilities, scored with proper scoring rules. It is therefore the capability detailed in full below, with empirical results reported in Section 6. The team season and player parametric families are trained estimators scored on the same leakage free holdout with proper metrics (Brier, log loss, MAE). However, on the present single-league, five-season sample they remain illustrative: they are included to show that the pipeline generalises across forecasting targets rather than to establish competitive accuracy. The remaining families are not fitted estimators. In fact, the deterministic indices, the counterfactual and similarity based reuse, and empirical frequency

tables carry no learned parameters or held out ground truth. Hence, they are assessed for routing correctness and output plausibility rather than predictive accuracy.

4.5.1 Match-Outcome Algorithm

Task and model. We frame match-outcome prediction as a three-class problem over $y \in \{H, D, A\}$ (home win, draw, away win).

The first base learner is a multinomial logistic regression over the standardized feature vector x:

$$P_{\mathrm{LR}}(y = k \mid x) = \frac{\exp(w_k^\top x + b_k)}{\sum_{j \in \{H,D,A\}} \exp(w_j^\top x + b_j)}, \qquad k \in \{H, D, A\}, \tag{4.1}$$

with class-specific weights w_k and bias b_k.

The second and third base learners, XGBoost and LightGBM, are both additive ensembles of M regression trees fit by gradient boosting. At round m, a tree h_m is fit to the negative gradient of the multiclass softmax loss:

$$\hat{y}_{i,k}^{(m)} = \hat{y}_{i,k}^{(m-1)} + \eta\, h_{m,k}(x_i), \qquad L = -\sum_i \sum_k y_{i,k} \log \frac{\exp(\hat{y}_{i,k})}{\sum_j \exp(\hat{y}_{i,j})}, \tag{4.2}$$

where η is the learning rate. XGBoost regularizes each tree's leaf structure directly in the split objective,

$$\Omega(h) = \gamma T + \tfrac{1}{2}\lambda \sum_{t=1}^{T} w_t^2, \tag{4.3}$$

with T leaves and leaf weights w_t Chen and Guestrin (2016); LightGBM optimizes the same loss but grows trees leaf-wise with gradient-based one-side sampling rather than XGBoost's depth-wise, histogram-based splitting Ke et al. (2017). The two disagree often enough on individual matches to be worth stacking instead of averaging.

Stacking. Each base learner $b \in \{\mathrm{LR}, \mathrm{XGB}, \mathrm{LGBM}\}$ outputs a probability vector $\hat{p}_i^{(b)}$ over the three classes for match i. These concatenate into a 9-dimensional meta-feature vector

$$z_i = \left[\hat{p}_i^{(\mathrm{LR})} \parallel \hat{p}_i^{(\mathrm{XGB})} \parallel \hat{p}_i^{(\mathrm{LGBM})}\right] \in \mathbb{R}^9, \qquad P(y = k \mid z_i) = \mathrm{softmax}(W_{\mathrm{meta}} z_i + b_{\mathrm{meta}})_k, \tag{4.4}$$

which a second logistic-regression meta-learner maps to a final probability Wolpert (1992). In order to keep the meta-learner from training on in-sample predictions, base-learner outputs are generated out-of-fold via 5-fold cross-validation. Each base learner trains on 4 folds and predicts on the held-out fifth, so every z_i comes from a model that never saw match i. This prevents the model from overfitting.

Calibration. The stacked ensemble's raw probabilities are recalibrated per class by isotonic regression, which fits a non-decreasing step function g_k minimizing

$$g_k^\star = \underset{g \text{ non-decreasing}}{\arg\min} \sum_i \left(\mathbb{1}[y_i = k] - g(\hat{p}_{i,k}) \right)^2 \tag{4.5}$$

on a held-out calibration fold Zadrozny and Elkan (2002), after which the three calibrated scores are renormalised to sum to one. This is crucial because a well-separated but poorly calibrated ensemble can rank outcomes correctly while overstating confidence. This is critical in practice: uncalibrated probabilities compared against market odds may imply value bets that are simply inexistent. This matters in practice, since a football fan comparing our probabilities against the industry's odds acts on the stated confidence, not just the ranking. Hence, the magnitudes must be trustworthy and not only their order.

Features. To avoid leakage, all pre-match features use only information available before kick-off: team-strength (Elo), rolling five-match form (points, goals for/against, shots, expected goals for/against), rest days, and team-level expected goals (xG) aggregated from player data. We train two variants: a with-odds model for offline analysis where historical odds exist, and a no-odds model for live prediction, where pre-match odds are unavailable.

Train/inference parity. For match-outcome, this is accomplished by running the same feature-engineering pipeline at inference time, producing a feature vector that must match the one seen during training. A synthetic fixture is appended to the historical match record and passed through an identical pipeline, so that rolling and xG features reflect a team's true pre-match state rather than an approximation. A runtime safety procedure refuses any feature vector that is incomplete rather than imputing it. Other families, described in Table 4.2, achieve parity by resolving the entity's latest row from the same feature frame used in training, applying the same runtime refusal on incomplete vectors.

Evaluation. The model is trained on 2020–2024 and evaluated out-of-sample on the held-out 2024–2025 season. We compare against two baselines: a base-rate predictor (training-set class frequencies) and the bookmaker's implied probabilities, computed by inverting average closing odds and normalising the three outcomes to sum to one.

5 Experiment Designs

5.1 Setup of single LLM agents

We evaluated two single-LLM conditions, both stripped of any multi-agent orchestration. Both share the same brief role description ("a Premier League data analyst answering only from the database") and the identical database schema given to every condition; they differ only in how the model reaches the data. In the single-LLM-with-tools condition, the model is given native function-calling access to the same two tools the autonomous MAS uses, run_sql for free-form queries and predict_match_outcome for fixture forecasts, and answers by issuing tool calls in a loop. In the single-LLM-with-raw-data condition, the model receives no tools and no database access; instead we inline a text dump of the core modelling tables (season standings, home/away splits, team and player season statistics) directly into the system prompt and instruct it to answer only from that data, stating explicitly when the required data is absent.

We chose to inline data rather than equip the model with retrieval tools because we wanted to isolate the LLM's data-reasoning capability from any data-retrieval or external-knowledge capability, and because a fixed, inlined dataset makes runs deterministic and directly comparable across models and architectures. By design and context size limitation the dump excludes the largest and most specialised tables (match events, player-match performance, transfers, market values, and injuries), so questions requiring those sources are unanswerable in this condition. This constraint accounts for its low accuracy on the corresponding query categories, as only 20 of the 30 eval questions can be answered with the data tables injected into the context window.

5.2 Ground truth dataset and evaluation process

Two evaluation sets are used in this report. The head-to-head MAS-versus-single-LLM comparison (Section 6.2) uses a re-scoped 60-question set (30 data-analysis, 30 prediction) run on the current multi-league database. The autonomy experiment (Section 6.1) and the memory experiments (Section 6.3) were conducted earlier on the original 46-question single-league set and are reported on that basis; their figures are therefore not directly comparable to the 60-question results (see Limitations, "Database substrate").

Our ground truth dataset comprises 60 questions spanning nine analytical categories (league outcomes, player- and team-season aggregates, home/away splits, xG, discipline, transfers/-market values, availability, match events, and edge cases) and four difficulty tiers, from simple lookups to multi-step edge cases. Of the 60 questions, 30 are data-analysis queries scored programmatically against deterministic SQL ground truth. The remaining 30 are prediction-related and evaluated differently: on probabilistic forecast quality (Brier score) and correct

delegation to the prediction model, rather than on an exact-match answer.

Each of the 30 data-analysis questions is paired with a hand-verified canonical SQL query whose execution against the database yields the reference answer; the questions, queries, and results are then content-hashed and frozen (together with the originating git commit) so that the agents never see the ground truth and the benchmark cannot drift between runs. At evaluation time we replay every question through each system configuration and score the produced answer programmatically against the frozen ground truth: exact-match with a small numeric tolerance for scalar answers, and row-set comparison for tabular and ranked answers. This is possible because each data-analysis item has a deterministic SQL ground truth. An LLM-as-judge (drawn from a different model family than the one under test) is integrated for genuinely free-text answers, but was not required for this golden set, since every question resolves either to a deterministic SQL result or to a numeric forecast score.

5.3 Golden-Set Questions (data analysis, $n = 30$)

The complete canonical SQL for each question is in `results/golden/ground_truth_v2.json`.

Table 5.1: Golden-set data-analysis questions ($n = 30$), with category, difficulty tier, and template coverage. IDs are inherited from the original 46-question set from which this benchmark was curated; gaps reflect questions not retained.

ID	Category	Difficulty	Cov	Question
Q01	team_league_outcomes	simple	in	Which team won the Premier League in 2024-2025?
Q02	team_league_outcomes	simple	in	Points Arsenal finished 2023-2024 with?
Q03	team_league_outcomes	medium	in	Full 2022-2023 PL final table (rank, team, points).
Q05	team_league_outcomes	medium	in	Best goal difference in 2024-2025?
Q06	team_league_outcomes	edge	out	Everton 2023-2024 points with vs without deduction?
Q07	team_league_outcomes	hard	out	Highest single-season points total, 2020-2025?
Q09	player_season_aggregates	simple	in	Top goalscorer in 2023-2024 and goals?
Q10	player_season_aggregates	medium	in	Top five goalscorers in 2024-2025 with totals.
Q11	player_season_aggregates	medium	in	Most assists in 2023-2024?
Q12	player_season_aggregates	edge	in	Cole Palmer total 2023-2024 goals (all clubs)?
Q16	home_away_splits	medium	in	Most home points in 2024-2025?
Q18	home_away_splits	hard	out	Biggest home-vs-away points gap in 2023-2024?
Q19	home_away_splits	simple	in	Arsenal home points in 2023-2024?
Q20	home_away_splits	medium	in	Most home goals in 2024-2025?
Q21	disciplinary	medium	in	Most yellow cards (team) in 2023-2024?

continued on next page

ID	Category	Difficulty	Cov	Question
Q23	disciplinary	medium	in	Most red cards (team) in 2024-2025?
Q24	disciplinary	medium	out	Total red cards league-wide in 2023-2024?
Q25	disciplinary	edge	out	PL matches refereed by Anthony Taylor in 2023-2024?
Q26	xg_performance	hard	out	Biggest xG overperformer (≥10 goals) 2023-2024?
Q27	xg_performance	hard	out	Biggest xG underperformer (≥10 goals) 2023-2024?
Q29	xg_performance	hard	out	Highest goals-to-xG ratio (≥15 xG) 2024-2025?
Q32	market_transfers	hard	in	Five largest incoming transfer fees to PL clubs?
Q33	market_transfers	medium	in	Most expensive incoming transfer, 2023-2024 window?
Q37	availability	hard	out	Most total days missed to injury, 2023-2024?
Q39	availability	medium	out	Injury spells recorded for Brentford players, 2023-2024?
Q41	event_player_match	medium	in	Most goals by one player in a single match, 2024-2025?
Q42	event_player_match	hard	out	Most fouls committed (summed) in 2023-2024?
Q44	event_player_match	medium	in	Biggest home win (margin) in 2024-2025?
Q45	edge_case	edge	in	Norwich City points in 2024-2025? (expect denial)
Q46	edge_case	edge	out	Matches in 2020-2021 with a recorded attendance figure?

Prediction questions ($n = 30$).

These are scored on (a) the Brier score of the predicted home/draw/away probabilities against the realised outcome, and (b) fidelity, namely whether the agent correctly delegated to the prediction model and reproduced its output. The 30 fixtures span 2024-08-16 to 2025-05-25 across 20 clubs, with a realised outcome balance of 10 home wins, 9 draws, and 11 away wins. Note that under argmax the model never assigns the draw the highest probability on any of the nine drawn fixtures (e.g. P04, P05, P07, P11), consistent with the draw-underprediction limitation discussed in Section 6.4.

Table 5.2: Prediction (forecast) questions ($n = 30$) with realised outcome and model ground-truth probabilities (H/D/A).

ID	Fixture	Date	Actual	Model GT (H/D/A)
P01	Manchester United vs Fulham	2024-08-16	H	0.557 / 0.178 / 0.265
P02	Arsenal vs Wolves	2024-08-17	H	0.800 / 0.184 / 0.016
P03	Everton vs Brighton	2024-08-17	A	0.239 / 0.224 / 0.538
P04	Nottingham Forest vs Bournemouth	2024-08-17	D	0.322 / 0.249 / 0.429
P05	Chelsea vs Crystal Palace	2024-09-01	D	0.460 / 0.200 / 0.341
P06	Southampton vs Manchester United	2024-09-14	A	0.214 / 0.191 / 0.595
P07	Brentford vs West Ham United	2024-09-28	D	0.454 / 0.278 / 0.268
P08	Leicester City vs Bournemouth	2024-10-05	H	0.278 / 0.217 / 0.505
P09	Fulham vs Aston Villa	2024-10-19	A	0.319 / 0.317 / 0.364
P10	Wolves vs Manchester City	2024-10-20	A	0.049 / 0.261 / 0.690
P11	Ipswich Town vs Leicester City	2024-11-02	D	0.462 / 0.310 / 0.229
P12	Arsenal vs Nottingham Forest	2024-11-23	H	0.655 / 0.212 / 0.132
P13	Newcastle United vs West Ham United	2024-11-25	A	0.627 / 0.200 / 0.174
P14	Newcastle United vs Liverpool	2024-12-04	D	0.244 / 0.234 / 0.522
P15	Aston Villa vs Manchester City	2024-12-21	H	0.426 / 0.190 / 0.384
P16	Tottenham Hotspur vs Liverpool	2024-12-22	A	0.349 / 0.245 / 0.406
P17	Brighton vs Brentford	2024-12-27	D	0.549 / 0.206 / 0.245
P18	West Ham United vs Fulham	2025-01-14	H	0.253 / 0.269 / 0.478
P19	Ipswich Town vs Brighton	2025-01-16	A	0.257 / 0.243 / 0.500
P20	Manchester City vs Chelsea	2025-01-25	H	0.605 / 0.211 / 0.184
P21	Crystal Palace vs Everton	2025-02-15	A	0.434 / 0.225 / 0.342
P22	Everton vs Manchester United	2025-02-22	D	0.415 / 0.249 / 0.336
P23	Nottingham Forest vs Arsenal	2025-02-26	D	0.247 / 0.234 / 0.519
P24	Fulham vs Tottenham Hotspur	2025-03-16	H	0.555 / 0.219 / 0.226
P25	Southampton vs Crystal Palace	2025-04-02	D	0.049 / 0.222 / 0.729
P26	Southampton vs Aston Villa	2025-04-12	A	0.233 / 0.194 / 0.573
P27	Tottenham Hotspur vs Nottingham Forest	2025-04-21	A	0.557 / 0.210 / 0.233
P28	Wolves vs Leicester City	2025-04-26	H	0.673 / 0.190 / 0.137
P29	Newcastle United vs Chelsea	2025-05-11	H	0.447 / 0.262 / 0.291
P30	Ipswich Town vs West Ham United	2025-05-25	A	0.301 / 0.229 / 0.469

5.3.1 Experiment 1: Testing different degrees of agent autonomy

First, we compared versions of the MAS that differ only in the autonomy of the Data Analyst agent; every other component (Manager, Data Discovery, Synthesizer) and the schema context were held identical. The fully controlled version can answer only by selecting from a set of pre-specified SQL templates covering common question types. The semi-controlled version has access to those same templates but may also write its own SQL, deciding per query which approach to take, and falling back to free SQL if a chosen template fails. The fully autonomous version writes all queries itself, generating two independent SQL statements and reconciling their results before answering. Because the semi-controlled version's action space is a superset of both others', it can replicate the controlled strategy by always choosing a template or the autonomous strategy by always writing SQL, so we hypothesised it would be at least as accurate as either. However, we note that this superset holds over the available actions, not the outcomes: the semi-controlled agent carries an additional routing decision (template vs. SQL) that is itself a potential source of error, so a larger action space does not by itself guarantee higher accuracy.

5.3.2 Experiment 2: Testing MAS vs both single LLM variants

Second, we tested whether the multi-agent orchestration itself adds value beyond simple tool access. We fixed the MAS at its best-performing autonomy level from Experiment 1 (fully autonomous) and compared it against two single-LLM baselines with no orchestration. The single-LLM-with-tools baseline is given native function-calling access to the same tools the MAS uses (run_sql and predict_match_outcome) and the identical schema context, isolating the effect of orchestration alone. The single-LLM-with-raw-data baseline receives no tools or database access; instead the core tables are inlined directly into its prompt, so it must answer purely by reasoning over the provided data. Comparing these three conditions lets us separate the gain from tool access (single-raw to single-tools) from the additional gain from multi-agent orchestration (single-tools to MAS).

5.3.3 Experiment 3: Testing memory augmented MAS

For the third experiment, we tested a memory-augmented MAS with an added persistent memory layer. We evaluate its contribution to answer quality and to per-query computational cost separately, since neither effect is visible on a golden set of independent questions. In fact, an unrelated query gives memory nothing to retrieve, and cost only diverges from a no-memory baseline once a cache or correction exists to reuse.

We therefore evaluate memory on short, ordered sessions rather than independent queries: 55 sessions in total, each isolating one of the three stores' specific mechanisms (20 for correction memory, 18 for SQL cache, 17 for conversation memory), with the same architecture running memory-disabled as the baseline arm in every case. Correction-memory sessions seed a known failure mode on the first turn and pose a related query on the second, testing whether a mistake written to correction memory is avoided on retrieval. Conversation-memory sessions use context-dependent follow-ups ("what about away?", "and the season before?") that cannot

be resolved without the prior turn. Separately, we stress-test the SQL-cache threshold itself on the 46-question golden set (Section 6), rather than in session form, since the near-miss failure mode it targets is a property of the threshold itself rather than of session-based reuse.

As described in section 4.2, memory is stored in PostgreSQL with the pgvector extension across three stores: a conversation memory of completed question–answer turns, an SQL cache of validated queries keyed by the originating question's embedding, and a correction memory of past failures and their modes. Text is embedded with all-MiniLM-L6-v2 and retrieved by cosine similarity against a per-store threshold. Thresholds differ by risk: SQL-cache reuse uses a high threshold because a cached hit directly answers the query, so a false match produces a confidently wrong result. Conversational and correction memories use lower thresholds, as they are injected only as supporting context the agent may disregard, making a loose match low-risk.

6 Results

6.1 MAS Autonomy Comparison

We first compared MAS versions with different degrees of autonomy, on the original 46-question single-league set (see the note on evaluation sets in Section 6.2). Since the SQL templates available to the controlled and semi-controlled agents are not helpful for the prediction-related queries, we limited this evaluation to the 46 data-analysis queries. We further split the dataset into in-coverage (28) and out-of-coverage (18) queries, indicating whether a query can be answered with an available SQL template or requires different query and join logic.

Table 6.1: Accuracy by degree of Data Analyst autonomy, split by SQL-template coverage (46 golden-set data-analysis questions; 28 in-coverage, 18 out-of-coverage).

Model	Variant	In-coverage	Out-of-coverage	Overall
Opus 4.8	Controlled	26/28 (93%)	1/18 (6%)	27/46 (59%)
Opus 4.8	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Opus 4.8	Autonomous	24/28 (85%)	17/18 (94%)	41/46 (89%)
GPT 5.5	Controlled	26/28 (93%)	3/18 (17%)	29/46 (63%)
GPT 5.5	Semi	25/28 (89%)	16/18 (89%)	41/46 (89%)
GPT 5.5	Autonomous	23/28 (82%)	16/18 (89%)	39/46 (85%)
Gemini 3.1 Pro Preview	Controlled	26/28 (93%)	2/18 (11%)	28/46 (61%)
Gemini 3.1 Pro Preview	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Gemini 3.1 Pro Preview	Autonomous	19/28 (68%)	13/18 (72%)	32/46 (70%)

For in-coverage queries the controlled version achieved the same accuracy of 93% (26/28) across all three frontier models. The semi-controlled variant with Opus 4.8 and Gemini 3.1 Pro Preview achieved the same accuracy of 93% for in-coverage queries. The autonomous version performed worse on in-coverage questions, with Opus 4.8 at 85% and GPT 5.5 at 82%.

For out-of-coverage queries the autonomous version of Opus 4.8 achieved the highest accuracy of 94%, while the controlled version achieved very low accuracy across all models, between 6–17%. The semi-controlled version achieved 89% for all three models.

Interestingly, the semi-controlled and controlled variants perform similarly for Opus 4.8 and GPT 5.5, while the autonomous variant of Gemini 3.1 Pro Preview collapses to 70%. This suggests that the fully autonomous variant is more model-sensitive than the semi-controlled superset.

The experiment clearly shows that limiting the agent to predefined templates leads to low accuracy on out-of-coverage queries, making it less useful for a production system exposed to unseen and ambiguous queries. While the overall highest accuracy was achieved by Opus 4.8 and Gemini 3.1 Pro Preview at 91%, we proceeded with the variant and model combination

that achieved the highest accuracy on out-of-coverage queries specifically, since we want to optimise the system for unseen and ambiguous queries. Hence, the subsequent experiments use a fully autonomous MAS with Opus 4.8.

6.2 MAS vs Single-LLM Agents

Next we tested the benefit of using a Multi-Agent System versus a single LLM. To isolate the orchestration benefit of our MAS, we defined two versions of the single LLM: one with equal tool access to the database and prediction function, and another with access only to the raw data. We compared the fully autonomous MAS against both single-LLM baselines.

Data-Analysis Accuracy, Tokens, and Latency

Table 6.2: MAS vs. single-LLM baselines on the 30 data-analysis golden-set questions.

Condition	Overall	Avg tokens	Avg latency
MAS (autonomous)	26/30 (87%)	32,324	16.7s
single-LLM + tools	26/30 (87%)	22,150	8.1s
single-LLM + raw data	21/30 (70%)	121,711	4.6s

Our results show that the MAS and the single-LLM-with-tools condition tie at 87% accuracy (26/30). The MAS consumes more resources for this identical accuracy: an average of 32,324 tokens and 16.7 seconds, against 22,150 tokens and 8.1 seconds for the single LLM. The single-LLM-with-raw-data condition reaches 70% (21/30) overall while consuming the most tokens, because the core data tables are inlined into the prompt and re-parsed on every call.

The raw-data figure must be read against a structural coverage limit rather than a reasoning gap. The inlined dump contains only the ENG-PL team- and player-season tables, so market/transfer, injury, event, and referee questions are unanswerable by construction. On the 20 answerable questions the raw-data condition scores 20/20; its lower overall total is driven entirely by the 10 questions whose source data is absent from the dump, where it correctly reports the data as unavailable.

The results therefore suggest that the largest accuracy differences trace to structured tool access and data coverage rather than to orchestration. Once tool access and schema context are held equal, multi-agent orchestration adds no measurable accuracy over a single tool-using LLM, at meaningfully higher token and latency cost.

Prediction Delegation

In addition to the data-analysis tasks, we tested a more complex query requiring delegation to our proprietary prediction models. The MAS and the single LLM with tools access these models through a function call; the single LLM with no tools has no access to the model.

Table 6.3: Delegation to the prediction agent: MAS vs. single-LLM baselines (30 prediction questions).

Condition	Avg Brier	Fidelity	Avg tokens	Avg latency
MAS (autonomous)	0.197	30/30	1,008	6.8s
single-LLM + tools	0.197	30/30	42,912	13.0s
single-LLM + raw data	0.196	0/30	1,204	7.7s

Both the MAS and the single LLM with tool access delegated correctly on all 30 fixtures and reproduced the model's forecast vector faithfully. Because both conditions perform single-pass parameter extraction and then call the identical `predict_match_outcome` function, their forecast outputs are identical by construction on all 30 fixtures; the prediction slice therefore admits no architectural comparison and no significance test. The single-LLM-with-tools condition reaches this identical output at far higher cost (42,912 tokens), exploring the database before delegating. The single LLM with no model access performed independent reasoning over outcome probabilities, scoring a marginally different Brier (0.196) that is statistically indistinguishable from the model's on this sample.

Paired Significance Test (McNemar)

To test whether the MAS outperforms the single LLM with equal tool access, we apply McNemar's exact test to the paired binary outcomes on the 30 data-analysis questions. We use the exact binomial form rather than the chi-square approximation, since the discordant count is below 10.

Overall, the two conditions disagree on only four questions ($b = 2$ MAS-only correct, $c = 2$ single-only correct), giving an exact two-sided p-value of 1.0: no significant difference. On the out-of-coverage subset ($n = 12$), where a MAS advantage was hypothesised, the direction in fact favours the single LLM ($b = 0$, $c = 2$, $p = 0.5$; single-LLM 12/12 vs MAS 10/12).

Manual inspection of the four discordant pairs (the questions McNemar's test counts, where the systems disagree) shows that two (Q33, Q45) are scorer artefacts, in which the single LLM was substantively correct but lost on answer formatting. That is, a transfer fee written as "116.6 million" rather than matched to 116600000, and a correct denial broken by markdown emphasis. The other two (Q25, Q42) are genuine disagreements, both MAS errors. The programmatic tie therefore slightly flatters the MAS.

With $n = 30$ and only two to four discordant pairs, this comparison is underpowered: it can neither confirm nor exclude a small true difference, and detecting an effect of the observed size would require roughly an order of magnitude more questions. What it does establish is that there is no evidence of a material accuracy advantage from orchestration on this workload.

6.3 Memory Augmented MAS Results

6.3.1 SQL Cache Threshold Stress Test

We test whether cache reuse is selective via a three tier sweep on the 46 question golden set. Each question is run cold, establishing a cache free baseline, then re-run as an exact repeat, a light reword, and a near miss (i.e. swapped player or season).

Table 6.4: Three tier SQL cache sweep on the 46 question golden set (0.88 cosine threshold).

Tier	Cache Hits	Accuracy	Avg sim	Avg Latency	Avg Tokens
Cold Baseline	0/46	72% (33/46)	n/a	14.44s	20,656
Exact Repeat	45/46	72%	0.998	11.97s	11,575
Light Reword	44/46	65% (3 drifted)	0.983	12.12s	11,339
Near Miss	43/46	∼7% (collapse)	0.978	12.10s	11,606

On true repeats and rewords the cache hits reliably (45/46, 44/46), cutting tokens 44% and latency 17% by skipping the SQL generation call, with no material accuracy loss on repeats. This is a concrete demonstration of how a functional memory layer cuts cost: average token usage falls from 20,656 on the cold baseline to 11,575 on an exact repeat, roughly 9,000 fewer tokens per query, while latency drops from 14.44s to 11.97s. In fact, the saving comes from skipping SQL generation entirely rather than from a cheaper generation, it scales directly with how often a workload reissues queries, the pattern typical of conversational analytics where users re-ask and refine. On light rewords, accuracy drops to 65% (3 of 46 drifted) despite a near identical hit rate, showing that a paraphrase level embedding match does not guarantee the reused SQL still answers the reworded question correctly. The threshold cannot catch this on its own: average similarity on light rewords (0.983) sits just below exact repeats (0.998) and far above the 0.88 cut off, so the score gives no signal that the reused SQL has stopped being correct and admits the reword with almost the same confidence as a true repeat.

Near misses expose a severe safety failure: cosine similarity is not reliable for temporal and entity distinctions. A near miss (same surface form, different team or season) still retrieves the cached result 43 times out of 46, and accuracy collapses to approximately 7%, since the cache confidently returns a wrong, previously cached answer with no error signal. Semantic similarity is good at finding candidate matches but bad at deciding whether they are safe to reuse. Raising the threshold from 0.88 to 0.99 could reduce false hits, but would also suppress legitimate rewords, so threshold tuning alone is not a sufficient fix.

This safety gap has since been addressed in the deployed system. Cache reuse is now properly gated by a lightweight entity consistency check applied on top of the embedding similarity threshold. Before a cached result is reused, discriminating tokens (season, metric family, and named entities) are extracted from the incoming query and compared against those of the cached query and its stored SQL. In addition, the reuse is rejected on any contradiction to make the system more robust. This preserves paraphrase hits while blocking the season and entity swap near misses that drove the previously noted collapse. The check is a cheap

textual comparison rather than an additional model call. Note that it adds negligible latency. However, it does not catch polarity swaps (e.g. "highest" vs. "lowest") that leave the entity, season, and metric unchanged. It must be acknowledged that the 46 question evaluation run predates this guard for the memory experiment. The session based cache experiment later ran with the guard active and showed no collapse, but those sessions posed only same entity, same season repeats, so they do not test the near miss blocking path. A rerun of the near miss sweep with the guard active is a straightforward next step, and the guard is designed to catch exactly the season and entity swaps that drove the collapse.

6.3.2 Session Based Memory Ablation

The stress test above isolated one store on a single-turn benchmark. Conversation and correction memory, however, only act across turns, so we evaluate them on short ordered sessions rather than independent queries. This allows memory to retrieve something. We run 55 sessions in total, each isolating one store's mechanism: 20 for correction memory, 18 for the SQL cache, and 17 for conversation memory. Every session is run twice under an identical architecture, once with memory enabled and once with it disabled, so the memory-off arm serves as a matched baseline and any difference is attributable to the store under test. This is the design of the session based memory ablation.

Table 6.5: Session based memory ablation: paired accuracy change from enabling each store, measured on beneficiary turns (memory on arm minus memory off arm). Positive values favour memory.

Store	Sessions	Δ accuracy	95% CI	Wilcoxon p
Correction memory	20	+0.200	[0.05, 0.40]	0.125
Conversation memory	17	+0.000	(-0.176, 0.162)	1.0
SQL cache	18	+0.000	n/a	n/a

Table 6.5 highlights the results of the memory ablation experiment. Each memory store is compared, to effectively demonstrate it's role the accuracy of the output of the MAS. Correction memory demonstrates a +0.200 accuracy improvement on later in session queries (95% CI [0.05, 0.40]; Wilcoxon signed rank $p = 0.125$, $n = 4$ nonzero session deltas of 20 sessions run). The bootstrap interval excludes zero, but the paired significance test does not reach the conventional threshold at this sample size; both should be read together rather than either alone.

The result is better read alongside its mechanism than either statistic alone. Of the 20 correction sessions, the induced failure recurred in the no memory arm in exactly 5, all involving a referee name column containing characters with hidden formatting that prevented the MAS from splitting on whitespace. This is a formatting artefact originating in the data sources. As a result, the naive equality filter misses silently, returning zero rows with no error thrown. In 4 of those 5, the memory arm had written a normalising pattern to correction memory after the first encounter and reapplied it correctly on the second. The fifth session shows the same miss in both arms, consistent with a retrieval miss below threshold. However, the retrieval

hit rate was not directly wired in this evaluation. The remaining 15 of 20 sessions produced identical outcomes in both arms, hence, no signal was diffused in either direction. This is a concrete demonstration that correction memory generalises to real world data quirks a fixed cleaning pass would not anticipate.

Conversation memory shows no net change in follow-up turns requiring prior-turn context (95% CI [-0.176, 0.162]; $p = 1$, $n = 17$ of 17 sessions). The zero highlighted here is not inertness but exact cancellations. Memory improved 5 sessions, degraded 5, and left 7 unchaged. Therefore, gains and losses were offset on this sample run. Secondary comparisons against the single-LLM baselines from Section 6.2 are more clear-cut: conversation memory substantially outperforms a stateless single LLM with no history (+0.441, 95% CI [0.265, 0.618], $p = 0.0005$, $n = 17$) but is beaten by a single LLM given the full conversation history in context (-0.559, 95% CI [-0.735, -0.382], $p = 0.0002$, $n = 17$). The full-history LLM resolves these follow-ups more reliably, so the case for retrieval-based conversation memory is one of cost and scalability rather than accuracy. Note that, carrying full history in context is far more token-expensive.

Accuracy was identical between cache on and cache off arms in all 18 session based cache tests (0 of 18 sessions showed any delta). This further explains how the cache is a cost mechanism rather than an accuracy one. Overall, the original memory hypothesis is therefore revised: the cache earns its place by cutting cost, not by necessarily improving accuracy.

6.4 Prediction Backtest Results

We evaluate the match-outcome model out-of-sample on the 379 fixtures of the held-out 2024–2025 Premier League season. The model is a calibrated stacking ensemble (logistic regression, XGBoost, and LightGBM base learners with a logistic meta-learner and isotonic calibration), trained on team-strength (Elo), recent form, and rest features. We compare two variants, without and with market odds as inputs, against a base-rate baseline (always predict the most frequent class) and the market's implied probabilities.

Table 6.6: Out-of-sample match outcome prediction, 2024–2025 season ($n = 379$).

Model	Accuracy	Brier Score
Base rate	40.9%	0.218
MA-FDSS (no odds)	53.3%	0.199
MA-FDSS (with odds)	53.6%	0.194
Market Implied	54.1%	0.193

The no-odds model improves 12.4 points over the base rate and lands within one point of the bookmaker despite using no market data. A bootstrap 95% confidence interval on its accuracy contains the market's 54.1%, making the two statistically identical. Adding odds yields only +0.3 points, implying the engineered features, team strength (Elo) above all, recover most of the market's win/draw/loss information. Group-wise permutation confirms Elo as dominant (-9.4 points when permuted, vs. -2.5 and -1.2 for defensive/attacking form).

7 Limitations

We report the following limitations in rough order of how much they bound the paper's central claims. The first two concern the interpretation of our headline result and should be read as scope conditions on it; the remainder concern specific components and evaluation choices.

Task scope does not exercise orchestration. Our data-analysis benchmark consists of single-step queries: each question maps to one retrieval and one answer. Multi-agent orchestration is theorised to help principally on *multi-step* tasks. For example, compound questions requiring decomposition, intermediate results that feed downstream stages, or planning over several dependent sub-queries. Our workload does not exercise this regime. The finding that orchestration adds no measurable accuracy over a single tool-using LLM should therefore be read as specific to single-step analytical querying, not as evidence that orchestration is unhelpful in general. Establishing whether a MAS earns its cost on genuinely multi-step, compositional queries is the most important experiment this work leaves open, and we expect it to be where any orchestration benefit, if one exists, would appear.

Tool access versus data coverage are confounded. We frame structured tool access as the dominant driver of accuracy (70% to 87%), but this comparison does not cleanly isolate tool access as a capability. The raw data condition and the tool using conditions both score 20/20 on the questions whose source data fits in the inlined dump. The raw data condition's shortfall arises entirely on the ten questions whose tables are too large to inline. The effect we measure is therefore better described as a context window scaling limit, since raw inlining cannot represent the full schema, than as evidence that tool access improves reasoning. A cleaner test would add a non-agent retrieval or text to SQL baseline with access to equivalent data coverage, isolating retrieval mechanism from information availability. We did not run this, and the causal interpretation of the 70 to 87 gap should be treated with corresponding caution.

The architecture is a static pipeline, not a dynamic MAS. Our system is a fixed orchestration graph (Manager, Data Discovery, Data Analyst, Synthesizer, Eval) executed as a largely sequential pipeline. It does not support the features associated with fully dynamic multi-agent systems: it cannot instantiate multiple agents of the same role in parallel to widen effective context, and it cannot flexibly re-route or revisit nodes based on intermediate state beyond the bounded retry loop. Our null result therefore speaks to *pipeline-style orchestration with specialised LLM stages*, and does not generalise to decentralised or dynamically-routed architectures where agents negotiate, spawn sub-agents, or revise the control flow at runtime. Whether those richer architectures would change the accuracy picture is untested here.

Small benchmark, single run. The data-analysis benchmark comprises 30 questions and the prediction benchmark 30 fixtures, small relative to comparable agentic-systems papers. Results derive from one run per condition, and the autonomous analyst generates SQL at non-zero temperature, so run-to-run variation of one to three questions is expected. The MAS-versus-single-LLM McNemar test rests on only two to four discordant pairs and is underpowered: it establishes the absence of a large orchestration effect but cannot exclude a small one. Repeated runs with variance estimates and an order-of-magnitude larger benchmark would be needed to resolve differences at the scale observed.

Prediction component is decoupled and its value-add is unestablished. The prediction package is architecturally separate from the multi-agent question: the agent-level evaluation tests only whether an agent routes correctly to the forecasting function, not any property of the MAS itself, and both tool-using conditions therefore produce identical forecasts by construction. On the 30-fixture sample the calibrated stacking ensemble (Brier 0.197) scores close to a single LLM reasoning over outcome probabilities without any model (0.196). We do not read this as evidence the model lacks value: the sample is small, and the fixtures are public Premier League results the language model may partly recall. But it does mean the model's added value over free-text estimation is not established here, and a larger, less memorisable test set would be needed to show it. The contribution here is the *integration* of a calibrated model as a first-class agent tool, not evidence that the model outperforms simpler alternatives. Under argmax the model also never predicts draws (roughly a quarter of fixtures), as draws lack a distinctive feature signature, and results are confined to a single Premier League season.

Database substrate changed after freezing. The evaluation database was repopulated from a single-league (ENG-PL) to a five-league schema after the original benchmark was frozen. The comparison reported here was re-scoped to ENG-PL on the current multi-league database; 29 of 30 re-scoped canonical queries reproduce the original single-league answers exactly. These figures are not directly comparable to earlier single-league results, and the autonomy experiment in particular was run on the original substrate.

Autonomy result is largely expected. The autonomy experiment's core finding, deterministic templates are reliable but cannot answer out-of-coverage questions, whereas free SQL generalises but is less reliable, is close to a truism. Its one less-obvious element is model-sensitivity: free-SQL reliability varied sharply by foundation model (Gemini's autonomous variant collapsed to 70% while Opus held), indicating that the expressiveness-versus-reliability trade-off is not uniform across models.

Scorer sensitivity. The answer-text scorer is sensitive to unit formatting and to markdown inside answer strings, which cost the single-LLM-with-tools condition two questions it answered substantively correctly. The scorer is applied identically to all conditions, so the primary comparison is unaffected in direction, but reported accuracies should be read as programmatic-match rates rather than substantive-correctness rates.

Memory benefit unproven on accuracy. The memory layer's benefit is unproven on accuracy: our golden set comprises independent queries, so retrieval has nothing prior to surface, and the correction mechanism, though functional, has no naturally recurring failure pattern to act on. Its demonstrated value is cost amortisation on repeated queries; the session-basedthe session-based evaluation shows no net conversational accuracy effect (per-session gains and losses offset), and a single LLM given full history in-context resolves the same follow-ups more reliably still, so the case for retrieval-based memory is one of cost and scalability rather than accuracy.

Cache safety. Similarity based reuse is unsafe for temporal data at the embedding threshold alone: near identical queries differing only by season embed at $\sim$0.99 and false hit, serving stale SQL. We therefore gate reuse with an entity consistency check layered on the similarity threshold, rejecting a hit when the season, metric, or named entities extracted from the incoming query contradict those of the cached query and its stored SQL. This closes the season and entity swap failures seen in the sweep, though it does not catch polarity swaps (e.g. "highest" vs. "lowest") that leave those tokens unchanged, and the near miss sweep has not yet been rerun under the guard to confirm the fix end to end. We also note that the correction store's write path conflates transient API failures with genuine errors, requiring a more discriminating error detector.

Modality and tooling. Our work is confined to textual data and a small tool set. A natural extension is a multimodal knowledge base and a broader tool catalogue to widen the range of answerable questions.

8 Conclusion

This project set out to determine whether, and when, a multi agent architecture is worthwhile for conversational football analytics. Our central empirical finding is a null result: on single step analytical queries, a single LLM with database tools matches a full multi agent system (both 87%, McNemar p = 1.0) while using roughly a third fewer tokens and half the latency. Orchestration is therefore not justified on accuracy grounds for this class of task, though its value on genuinely multi step, compositional queries, which our benchmark does not exercise, remains open and is the natural next experiment.

What most separates the conditions is not orchestration but access to structured data: raw in context inlining cannot scale to a full schema, making a clean, documented, queryable data layer the higher leverage investment. Where a multi agent pipeline earns its place is engineering robustness, error isolation and inspectable control flow, rather than measured accuracy.

The memory layer told the same story: it proved a cost and scalability mechanism rather than an accuracy one, with conversation memory showing no net accuracy gain over a no-memory baseline and itself bettered by a single LLM carrying the full history, while similarity based cache reuse required an explicit entity consistency guard to stay safe on temporal queries.

On prediction, the two architectures are identical by construction, since both reduce to single pass parameter extraction plus a function call; the contribution is the integration of a calibrated match outcome model as a first class agent tool. On our 30 fixture sample its Brier score is close to that of a single LLM reasoning without a model, though this small sample and the public, memorisable nature of the fixtures limit what can be concluded about the model's added value.

A Appendix: Experimental Setup and Results

This appendix provides the model configuration, the exact prompts used for each condition, the per-question correctness matrices, and the failure taxonomy and root-cause analysis behind the results in the main text. The golden-set and prediction questions are listed in Table 5.1 and Table 5.2 in the main text and are not reproduced here; references below use question IDs from those tables.

A.1 Model IDs and Configuration

Table A.1: Model identifiers and temperature configuration per logical family.

Logical family	LiteLLM model ID	Temperature
claude	`anthropic/claude-opus-4-8`	model default (omitted)
gpt	`gpt-5.5`	model default (omitted)
gemini	`gemini/gemini-3.1-pro-preview`	model default (omitted)

Routing. All model calls go through LiteLLM (`completion`), giving one OpenAI-style interface (including cross-provider function-calling) across the three families. **Temperature.** All three frontier models deprecate or restrict the `temperature` parameter, so we omit it and let each model use its own default. SQL-generating paths request 0.3, but this is suppressed for these models; this is a methodological constraint rather than a fixed temperature. **Max tokens.** 4096, generous enough that thinking models return content rather than only reasoning. **Memory.** Disabled for all evaluation runs (no read or write) so the comparison isolates orchestration. **Tokens and latency.** Captured locally via LiteLLM usage accounting per run, except MAS data-analysis token totals, which were recovered from Langfuse traces because local capture does not see the data-analyst self-debate loop (see Section A.5).

A.2 Failure Taxonomy (semi-controlled variant)

A = chose a template when SQL was needed; B = wrong template or params; C = template failed then SQL fallback, still wrong; D = free-SQL generation wrong; E = errored or unclassified.

Table A.2: Failure taxonomy for the semi-controlled variant, by model and failure mode (autonomy experiment, $n = 46$).

Code	Failure mode	claude	gpt	gemini	Total
A	chose template when SQL needed	0	0	0	0
B	wrong template / params	2	3	2	7
C	template failed → SQL fallback, wrong	0	0	0	0
D	free-SQL generation wrong	2	2	2	6
E	errored / unclassified	0	0	0	0

A.3 Root-Cause Analysis of Failures

These analyses concern the autonomy experiment ($n = 46$, single-league set; Section 6.1).

Controlled variant collapses on out-of-coverage (6–17%). By construction the template catalogue (Section A.4.13) cannot express computed columns, `HAVING` filters, cross-season aggregation, or the events, transfers, and injuries tables. Every out-of-coverage miss is a no-template-fits failure, the expected result of constraining the agent to templates and the central motivation for allowing free SQL. **Universal failures (all variants, all models).** Two questions are missed by every configuration. Q41 (most goals by one player in a single match): `player_match_performance` has a per-(player, game) grain that all systems mis-aggregated, and the matching template returns a different shape than the ground truth expects; this is a likely ground-truth grain mismatch worth re-auditing. Q25 (matches refereed by Anthony Taylor): the `referee` column contains a non-breaking space (`chr(160)`) that the canonical SQL normalises with `replace(...)`; agents that wrote the natural `referee = 'Anthony Taylor'` returned zero rows. This is a data-quality landmine rather than a reasoning failure, and single-LLM-with-tools alone recovered it.

Autonomous loses some in-coverage questions. On questions a template would have answered cleanly, free SQL occasionally diverges on table formatting, ordering, and aggregation choices. For Gemini the autonomous self-debate loop frequently failed to converge, the main driver of its 70% overall. This is the cost of autonomy: more expressive, but with more ways to be subtly wrong.

Semi-controlled failures split evenly. Seven are wrong-template-or-param and six are wrong-SQL. Bucket A is empty, so no model ever picked a template when SQL was strictly required; the routing decision itself is sound and the errors are downstream.

Single-LLM-with-raw-data is bounded twice over. First, a coverage ceiling: only the core tables are inlined, so questions needing tables too large to inline are unanswerable by construction. Second, a reasoning shortfall within reach: even among the answerable questions the gap concentrates in aggregations over the largest inlined table.

A.4 Exact Prompts per Condition

All conditions share the same role preamble and schema context (`get_schema_context()`), injected where indicated by `{schema}`. Variable interpolations are shown in `{braces}`.

A.4.1 Shared preamble (all agents)

```
You are a Premier League data analyst. Answer the user's question using
ONLY the database. Rules: seasons are 'YYYY-YYYY' (hyphen, e.g.
'2023-2024'); the latest season is 2024-2025 (no 2025-26 data); point
deductions are already in the points columns; prefer IDs over names for
joins; canonical team names (e.g. 'Manchester United', 'Wolves').
```

A.4.2 MAS Manager (intent + entity extraction)

```
{preamble}
Classify and extract for this Premier League question.
Question: {question}
Respond JSON: {"intent": "data_request"|"prediction_match"|"conversational",
"seasons": ["YYYY-YYYY", ...], "teams": [...], "metrics": [...],
"home_team": "...", "away_team": "...", "match_date": "YYYY-MM-DD"}.
Use intent='prediction_match' ONLY when asked to forecast a specific
fixture's outcome; then fill home_team, away_team and match_date.
Resolve 'this/current season' to 2024-2025 (latest data).
```

A.4.3 MAS Data Discovery

```
{preamble}
Database schema:
{schema}
Question: {question}
Manager context: {manager_json}
Respond JSON: {"relevant_tables": [...], "notes": "short join/grain guidance"}.
```

A.4.4 MAS Data Analyst, CONTROLLED (template only)

```
{preamble}
You can ONLY answer using these templates:
{template_catalogue}
Question: {question}
Respond with JSON: {"template_name": "...", "params": {"season": "...",
"team": "...", "metric": "...", "top_n": 50}}. Include only the params
the template needs.
```

On template error: one corrective retry with `(Previous attempt failed: {error})` appended.

A.4.5 MAS Data Analyst, SEMI (route template-or-SQL)

```
{preamble}
You may either USE A TEMPLATE or WRITE CUSTOM SQL.
Templates:
{template_catalogue}
```

```
Question: {question}
Respond JSON: {"approach": "template"|"sql", "template_name": "...",
"params": {...}} (if template). If "sql", just {"approach":"sql"}.
```

If approach = "sql", or a chosen template errors, control passes to the self-debate SQL loop.

A.4.6 MAS Data Analyst, AUTONOMOUS (free SQL + self-debate)

Two independent candidates are generated per iteration (up to 2 reconcile rounds); their result sets must agree or the answer is flagged low-confidence.

```
# Candidate A
{preamble}
Database schema:
{schema}
Question: {question}
Write the most direct correct query.
Return ONLY one SQL SELECT statement, no prose, no code fences.

# Candidate B
... Write an independent query (different phrasing/joins) for the same answer.

# Reconcile round (if A and B disagree)
... Your two previous SQL attempts DISAGREED on their results -- reconcile them.
Attempt A: {sql_a}
Attempt B: {sql_b}
Find the correct query.
```

A.4.7 MAS Synthesiser (data questions)

```
{preamble}
Question: {question}
Query result (JSON):
{first_30_rows}
Answer the question directly and concisely using ONLY this result.
State numbers exactly.
```

A.4.8 MAS Prediction synthesiser

```
{preamble}
Question: {question}
The match-outcome model returned (probabilities):
{prediction_json}
Answer the question using these probabilities. State the three
probabilities (home win / draw / away win) explicitly and the most
likely outcome.
```

A.4.9 Single-LLM + tools (system prompt)

Native function-calling; tools = `run_sql`, `predict_match_outcome`; max 8 steps.

```
{preamble}
Use the provided tools to answer: ‘run_sql‘ to query the database, and
‘predict_match_outcome‘ to forecast a fixture’s result. Then answer.
```

```
Database schema:
{schema}
```

A.4.10 Single-LLM + raw data (system prompt)

No tools; the four core tables are inlined as CSV.

```
{preamble}
You have NO tools and NO database access. Answer ONLY from the schema
and the inlined data below. If the data needed is not present, say so.
Schema:
{schema}
=== INLINED DATA ===
{csv_dump_of: standings, home_away, team_discipline, player_season}
```

A.4.11 Single-LLM + raw data (prediction variant)

```
{preamble}
You have NO tools and NO prediction model -- forecast using ONLY the
recent results below. Reason about form, then give CALIBRATED
probabilities for {home_team} (home) vs {away_team} on {match_date}
that sum to 1.0.
=== RECENT RESULTS ===
{home_team and away_team's last 10 matches strictly before match_date}
End your answer with EXACTLY this line:
PROBABILITIES: home=<p>, draw=<p>, away=<p>
```

A.4.12 Tool: predict_match_outcome (schema shown to the model)

```
name: predict_match_outcome
description: Predict probabilities of each outcome (home win / draw /
 away win) for a Premier League fixture, using the trained match-outcome
 model. Returns three probabilities summing to 1.0 plus the model's
 confidence. Provide the match_date (the model is causal -- it uses only
 data before that date).
input: { home_team (req), away_team (req), match_date 'YYYY-MM-DD' (req),
 season (opt) }
```

A.4.13 Template catalogue (controlled / semi)

```
- league_table: Full final league table for a season. (season)
- team_season_summary: One team's season summary incl. home/away points
  & goals, possession. (team, season)
- home_away_table: Teams ranked by a home/away split metric.
  (season; metric in home_points|away_points|home_wins|away_wins|
   home_goals_for|away_goals_for)
- player_season_leaders: Top players by a season stat.
  (season; metric in goals|assists|minutes|yellow_cards|red_cards|shots)
- player_team_leaders: Top players at a club by a season stat.
  (team, season; metric in goals|assists|minutes)
- team_stat_leaders: Teams ranked by a team-season stat.
  (season; metric in yellow_cards|red_cards|goals|shots|possession)
- head_to_head: All matches between two teams in a season.
```

```
  (team_a, team_b, season)
- match_results: Matches in a season ordered by home win margin. (season)
- market_value_leaders: Players ranked by latest market value. (none)
- market_value_team_leaders: Players at a club ranked by latest value. (team)
- player_match_leaders: Top single-match player performances.
  (season; metric in match_goals|match_assists)
- transfer_leaders: Largest incoming transfer fees to PL clubs. (none)
```

A.5 Caveats

Single-repeat ($n = 30 + 30$); no variance estimates, so differences of one to three questions are within noise. MAS data-analysis token totals come from Langfuse; all other token totals come from local capture, so the two are not directly comparable across tables. Q41 and Q25 ground truth should be re-audited (grain mismatch and non-breaking-space normalisation).

References

Rory P. Bunker and Fadi Thabtah. A machine learning framework for sport result prediction. *Applied Computing and Informatics*, 15(1):27–33, 2019.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pages 785–794, 2016.

Anthony C. Constantinou. Dolores: A model that predicts football match outcomes from all over the world. *Machine Learning*, 108(1):49–75, 2019.

Tom Decroos, Lotte Bransen, Jan Van Haaren, and Jesse Davis. Actions speak louder than goals: Valuing player actions in soccer. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1851–1861, 2019.

Antonio Gulli. *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*. 2025.

Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, and Chenglin Wu. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

Ditsuhi Iskandaryan et al. The effect of weather in soccer results: An approach using machine learning techniques. *Applied Sciences*, 10(19), 2020.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 3146–3154, 2017.

Xinyi Li et al. A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges. *arXiv preprint arXiv:2402.01680*, 2024.

Jiayuan Rao et al. Towards universal soccer video understanding with a multi-agent system. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, 2025.

Robert Rein and Daniel Memmert. Big data and tactical analysis in elite soccer: Future challenges and opportunities for sports science. *SpringerPlus*, 5(1):1410, 2016.

Alessio Rossi, Luca Pappalardo, Paolo Cintia, F. Marcello Iaia, Javier Fernández, and Daniel Medina. Effective injury forecasting in soccer with gps training data and machine learning. *PLOS ONE*, 13(7):e0201264, 2018.

Edward Wheatcroft and Ela Sienkiewicz. Forecasting and evaluating football match outcomes using probabilistic scoring rules. *International Journal of Forecasting*, 2021.

David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

Andrew Wong et al. Forecasting english premier league match outcomes using stacking and voting ensembles. *Expert Systems with Applications*, 2024.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *arXiv preprint arXiv:2308.08155*, 2023.

Hao Yang et al. Multi-agent systems powered by large language models: Applications, advantages, and challenges. *arXiv preprint*, 2024.

Rui Ye et al. MAS-GPT: Training LLMs to build multi-agent systems in a single inference. *arXiv preprint*, 2025.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921, 2018.

Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '02)*, pages 694–699, 2002.